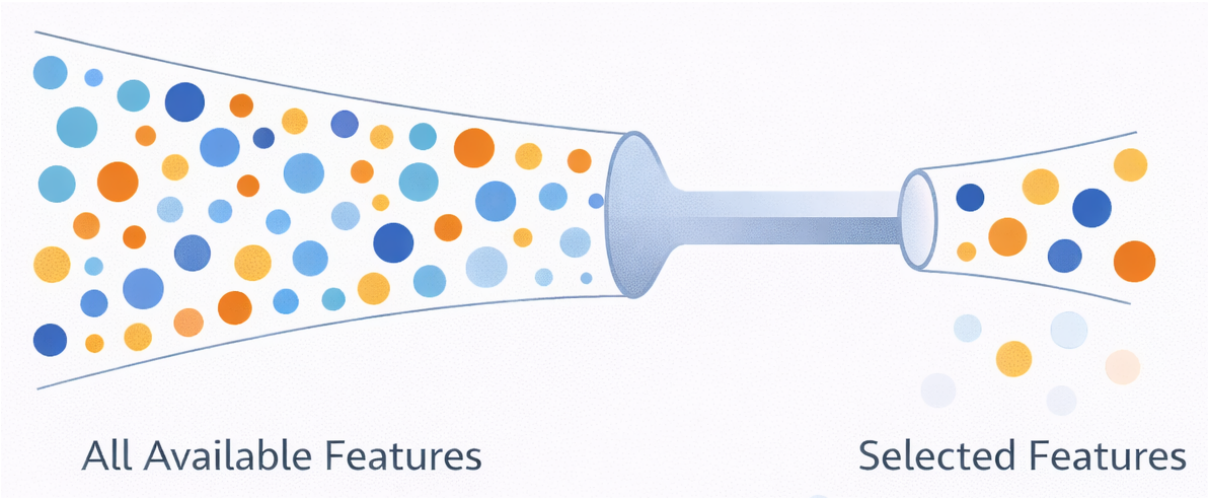
Chapter 16: Feature Selection

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to distinguish between causal feature selection (focused on coefficient validity) and predictive feature selection (focused on out-of-sample performance)
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to apply filter methods (variance thresholds, univariate tests, correlation analysis) for initial feature screening
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to implement wrapper methods (RFECV, sequential feature selection) to identify optimal feature subsets through model-based evaluation
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to calculate and interpret permutation feature importance and VIF for understanding feature contributions
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to integrate feature selection safely into scikit-learn pipelines to prevent data leakage

16.1Introduction



Chapter Philosophy: *Feature selection is not one technique—it is a mindset that differs fundamentally depending on your project goal. Causal inference projects remove features to ensure interpretable coefficients. Predictive inference projects remove features to improve out-of-sample performance. Using the wrong approach for your goal can undermine your entire model.*

Feature selection The process of choosing which available features to include in a model and which to remove in order to improve model performance, prevent overfitting, or ensure valid interpretation of results. is the process of choosing which available features to include in a model and which to remove. Every dataset contains features that are redundant, noisy, or misleading. Including all of them can cause a model to overfit the training data, perform poorly on new observations, or produce coefficients that cannot be trusted. The goal of feature selection is to keep the features that genuinely help and remove the ones that hurt.

However, the word “help” means very different things depending on whether you are building a predictive model or an explanatory model. In a predictive project, a feature helps if it improves out-of-sample accuracy. In a causal project, a feature helps if it clarifies the relationship between an independent variable and the outcome. These two definitions lead to fundamentally different selection strategies, different techniques, and different conclusions—even on the same dataset.

This chapter teaches both approaches. We will learn automated techniques available in scikit-learn for predictive feature selection—from quick, pre-model filters through model-based wrapper and embedded methods, culminating in permutation feature importance. We will also revisit how feature selection works in causal and explanatory modeling using VIF scores and domain-driven reasoning. By the end, you will know which approach to use, when, and why.

Learning Objectives

1. Understand why feature selection differs for causal versus predictive projects

2. Distinguish between filter, wrapper, and embedded selection methods
3. Apply SelectKBest, RFECV, SelectFromModel, and SequentialFeatureSelector in scikit-learn
4. Use permutation feature importance as a model-agnostic importance metric
5. Apply VIF-based removal for causal and explanatory models
6. Integrate feature selection into scikit-learn pipelines to prevent data leakage

Feature Selection vs. Feature Extraction

Before we begin, it is important to clarify what feature selection is *not*. Feature selection keeps original variables and removes some of them. *Feature extraction* (also called dimensionality reduction) creates entirely new variables from combinations of the originals—techniques like PCA, t-SNE, and UMAP fall into this category. Feature extraction can improve predictive performance, but the new variables are mathematical constructs that are difficult or impossible to interpret causally. You cannot make a causal claim about "Principal Component 2." For this reason, feature extraction is covered separately in the next book. In this chapter, every technique preserves the original features so that results remain interpretable.

Where Feature Selection Fits in the Pipeline

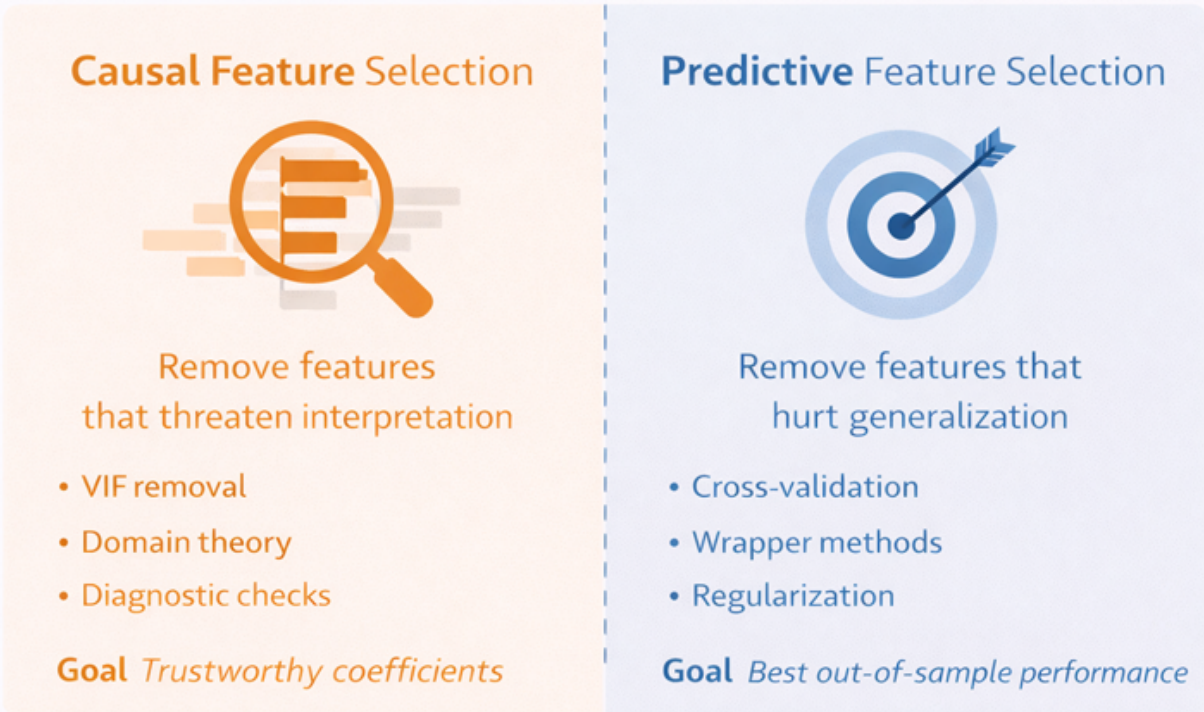
Feature selection belongs in the *Modeling* phase of the pipeline, after data preparation and exploration but before final model training and evaluation. It sits between having clean, engineered features and choosing a final algorithm. Done well, it reduces overfitting, speeds up training, and produces

simpler models that generalize better. Done poorly—or skipped entirely—it can produce models that memorize noise rather than learn signal.

Feature selection is most valuable when the number of features is large relative to the number of observations—a situation sometimes called the *curse of dimensionality*. When features vastly outnumber samples (for example, 2,000 features with only 200 observations), models are prone to overfitting: they can find spurious patterns in the training data that do not generalize to new observations. Removing irrelevant features forces the model to focus on genuinely informative variables. Conversely, when the sample-to-feature ratio is very favorable (for example, 100,000 observations with 15 features), feature selection is less likely to improve performance—the model already has enough data to learn which features matter and which to ignore.

Throughout this chapter, we continue using the Lending Club dataset from the prior chapters. This continuity lets us build on the preprocessing and modeling work already completed and compare results directly.

16.2Two Paradigms



The most important decision in feature selection is not which technique to use—it is which *paradigm* you are operating in. The same dataset can require completely different feature selection strategies depending on whether you are asking a causal question or a predictive question.

Causal Feature Selection (Explanatory Modeling)

Goal: Ensure that model coefficients can be interpreted as valid estimates of each feature's effect on the outcome.

Philosophy: "Remove features that threaten interpretation."

When building an explanatory model—for example, "What borrower characteristics drive loan default rates?"—the primary concern is coefficient validity, not predictive accuracy. Features are removed when they introduce multicollinearity (inflating or distorting coefficients), when they are theoretically inappropriate (mediators on the causal path, colliders, or

proxies for the outcome), or when diagnostics such as residual analysis reveal specification problems.

The primary diagnostic tool is the *variance inflation factor* (VIF). A VIF score measures how much a feature's variance is inflated by its correlation with all other features. The standard thresholds are:

- **VIF below 3:** Very conservative threshold sometimes used in academic research and regulatory settings where coefficient precision is critical.
- **VIF below 5:** Generally acceptable—multicollinearity is not distorting coefficients meaningfully.
- **VIF between 5 and 10:** Moderate concern—coefficients may be unstable. Investigate further.
- **VIF above 10:** Severe multicollinearity—coefficients are likely unreliable. Consider dropping the feature.

These thresholds are conventions, not laws. Some domains use stricter cutoffs (VIF above 4), while others tolerate higher values when theory strongly supports including a feature. The key principle is that in causal modeling, you do *not* optimize for predictive accuracy. Dropping a feature that improves R^2 but introduces multicollinearity is the wrong trade.

Predictive Feature Selection (Machine Learning)

Goal: Maximize out-of-sample predictive performance.

Philosophy: "Remove features that hurt generalization."

When building a predictive model—for example, "Which loans will default in the next quarter?"—the primary concern is how well the model performs on data it has never seen. Features are removed when dropping them

improves validation or test metrics, when they add noise without signal, or when they increase computational cost without meaningful predictive benefit.

In predictive modeling, multicollinearity is not inherently a problem. If two highly correlated features both improve predictions, you can keep both. You do *not* care about interpreting individual coefficients—you care about the model's aggregate output. The selection criterion is always cross-validated performance.

Quick Comparison

Table 16.1

	Causal / Explanatory	**Predictive / ML**
Remove when...	Feature threatens coefficient interpretation	Feature hurts out-of-sample accuracy
Primary tools	VIF, domain theory, residual diagnostics	Cross-validation, wrapper methods, regularization
Multicollinearity	Must be addressed	Often irrelevant
Evaluation metric	Coefficient validity, R^2, residual patterns	Test accuracy, F1, AUC, log loss
Overfitting concern	Specification error, omitted variable bias	Memorizing noise, poor generalization

The rest of this chapter is organized around the predictive paradigm first (Sections 3–6) because it aligns with the automated sklearn techniques. We then return to the causal paradigm (Section 7) to cover VIF and domain-driven selection. The chapter concludes by showing how to integrate feature selection into pipelines safely and a case study comparing both approaches on the same dataset.

16.3Filter Methods (Pre-Model Selection)

Filter methods evaluate features *before* training a model. They are fast, scalable, and useful as a first pass to eliminate obviously weak features. However, because they evaluate each feature independently (or in simple pairwise comparisons), they cannot detect complex interactions between features. Think of filter methods as a coarse screen that removes debris before the fine-tuning begins.

Data Setup

We will reuse the same Lending Club preprocessing established in prior chapters. The code below loads the data, engineers features, and prepares the label. One important addition: we now drop *outcome-adjacent* features—columns like *total_pymnt* and *total_rec_prncp* that are only known after a loan has been active for some time. In the ensemble chapter, these features dominated importance rankings because they are effectively proxies for the outcome. But if we want to generate a prediction that will help loan officers make decisions, we must first exclude features that would not be available at prediction time.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

df = pd.read_csv(&quot;lc_small.csv&quot;)

# --- Drop columns not useful for modeling ---
drop_cols = [&quot;loan_status_numeric&quot;, &quot;emp_title&quot;,
&quot;title&quot;, &quot;earliest_cr_line&quot;, &quot;issue_d&quot;]
df = df.drop(columns=[c for c in drop_cols if c in df.columns])

# --- Drop outcome-adjacent features (not available at origination) ---
outcome_adjacent = [&quot;total_pymnt&quot;, &quot;total_rec_prncp&quot;,
&quot;total_rec_int&quot;, &quot;total_rec_late_fee&quot;]
df = df.drop(columns=[c for c in outcome_adjacent if c in df.columns])
```

```
      # --- Convert term to numeric ---
      if &quot;term&quot; in df.columns:
        df[&quot;term&quot;] = df[&quot;term&quot;].astype(str).str.extract(r&quot;
(\d+)&quot;).astype(float)

      # --- Convert emp_length to numeric ---
      if &quot;emp_length&quot; in df.columns:
        emp = df[&quot;emp_length&quot;].astype(str).str.strip()

        emp = emp.replace({&quot;nan&quot;: np.nan, &quot;None&quot;: np.nan,
                           &quot;10+ years&quot;: &quot;10&quot;, &quot;&lt; 1
year&quot;: &quot;0&quot;})

        df[&quot;emp_length_years&quot;] = pd.to_numeric(emp.str.extract(r&quot;
(\d+)&quot;)[0], errors=&quot;coerce&quot;)
        df = df.drop(columns=[&quot;emp_length&quot;])

      # --- Handle informative missingness ---
      for col in [&quot;mths_since_last_delinq&quot;, &quot;mths_since_last_record&quot;]:
        if col in df.columns:
          df[col + &quot;_missing&quot;] = df[col].isna().astype(int)
          fill_val = df[col].max(skipna=True) + 1 if pd.notna(df[col].max()) else 0
          df[col] = df[col].fillna(fill_val)

      # --- Create binary label ---
      bad_statuses = {&quot;Charged Off&quot;, &quot;Default&quot;}
      df[&quot;loan_good&quot;] =
(~df[&quot;loan_status&quot;].isin(bad_statuses)).astype(int)
      y = df[&quot;loan_good&quot;].copy()
      X = df.drop(columns=[&quot;loan_status&quot;, &quot;loan_good&quot;]).copy()

      # --- Identify column types ---
      cat_cols = X.select_dtypes(include=[&quot;object&quot;,
&quot;category&quot;]).columns.tolist()
      num_cols = X.select_dtypes(include=[&quot;number&quot;]).columns.tolist()

      print(f&quot;Features: {X.shape[1]} ({len(num_cols)} numeric, {len(cat_cols)}
categorical)&quot;)
      print(f&quot;Observations: {X.shape[0]}&quot;)
      print(f&quot;Label balance: {y.mean():.3f} good, {1 - y.mean():.3f} bad&quot;)


      # Output:
      # Features: 27 (22 numeric, 5 categorical)
      # Observations: 10476
      # Label balance: 0.914 good, 0.086 bad
```

We now have 27 features (22 numeric, 5 categorical) and a binary label. The dataset is imbalanced—about 91% of loans are in good standing—which we will keep in mind throughout the chapter.

Missing Value Ratio

The simplest filter: if a feature is mostly missing, it carries very little information. A common threshold is to drop features with more than 50% missing values. We already handled the two high-missingness columns (*mths_since_last_delinq* and *mths_since_last_record*) by creating indicator variables and filling the originals. But in larger datasets, this check can eliminate entire columns before any modeling begins.

```
# Check missing value ratios
missing_pct = X.isnull().mean().sort_values(ascending=False)
print(&quot;Features with any missing values:&quot;)
print(missing_pct[missing_pct &gt; 0].to_string())

# Drop features above a threshold (50% missing)
high_missing = missing_pct[missing_pct &gt; 0.50].index.tolist()

if high_missing:
  print(f&quot;\nDropping {len(high_missing)} features with &gt;50% missing:
{high_missing}&quot;)
  X = X.drop(columns=high_missing)
else:
  print(&quot;\nNo features exceed 50% missing threshold.&quot;)


# Output:
# Features with any missing values:
# emp_length_years    0.069588
# revol_util          0.001623
# dti                 0.000955
# No features exceed 50% missing threshold.
```

To be clear, we likely would have addressed missing values earlier in the data preparation process. But for the sake of this chapter, I wanted to point out that excessive missing data is an obvious sign of a problem that should be addressed before modeling.

Low Variance Filter

Features with very low variance contribute almost no information because they are nearly constant. Scikit-learn provides VarianceThreshold to remove

such features automatically. This filter works on numeric features only and is applied before scaling (since scaling would change the variance).

```
from sklearn.feature_selection import VarianceThreshold

# Apply only to numeric columns (categorical variance is not meaningful here)
X_num = X[num_cols].copy()

# Default threshold=0 removes only constant features
# A small threshold like 0.01 removes near-constant features
vt = VarianceThreshold(threshold=0.01)
vt.fit(X_num)
low_var = X_num.columns[~vt.get_support()].tolist()

print(f&quot;Near-constant features (variance &lt; 0.01): {low_var}&quot;)
print(f&quot;Features remaining: {vt.get_support().sum()} of {X_num.shape[1]}&quot;)


# Output:
# Near-constant features (variance &lt; 0.01): ['acc_now_delinq']
# Features remaining: 21 of 22
```

The feature *acc_now_delinq* (accounts currently delinquent) has almost no variance because the vast majority of borrowers have zero current delinquencies. A feature that is nearly the same value for every observation cannot meaningfully distinguish between loan outcomes.

Univariate Statistical Tests: SelectKBest

Univariate filters test the relationship between each feature and the label independently using a statistical test. Scikit-learn's SelectKBest selects the top *k* features based on a chosen scoring function. The scoring function depends on your data types:

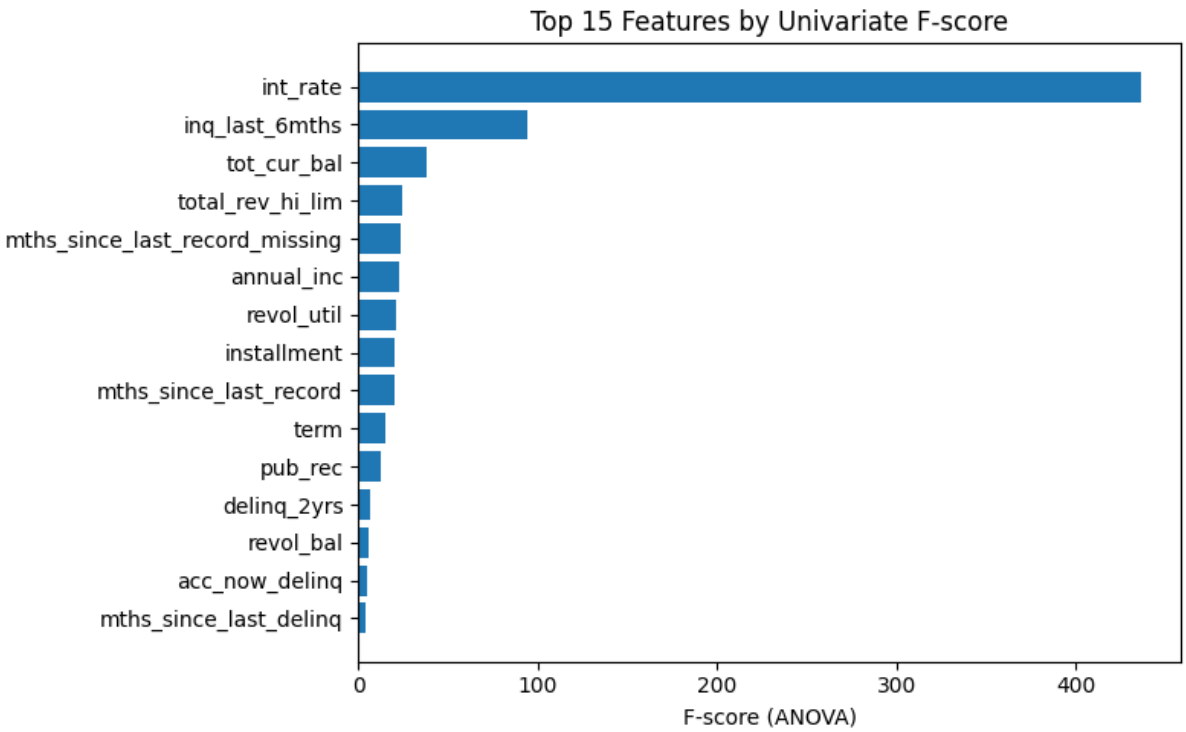
- **f_classif:** ANOVA F-statistic—use when features are numeric and the label is categorical
- **f_regression:** Pearson correlation F-statistic—use when features are numeric and the label is numeric

- **chi2:** Chi-squared statistic—use when both features and label are categorical (features must be non-negative)
- **mutual_info_classif / mutual_info_regression:** Mutual information—captures non-linear relationships but is slower to compute

Since our Lending Club features are numeric (after preprocessing) and our label is binary categorical, *f_classif* is the appropriate default. Let's use SelectKBest to rank all numeric features and visualize the top performers.

```
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.impute import SimpleImputer
import matplotlib.pyplot as plt

# Impute missing values for univariate testing (median for numeric)
X_num_imputed = pd.DataFrame(
  SimpleImputer(strategy=&quot;median&quot;).fit_transform(X[num_cols]),
  columns=num_cols, index=X.index
)

# Fit SelectKBest using all features (k='all') to see every score
skb = SelectKBest(score_func=f_classif, k=&quot;all&quot;)
skb.fit(X_num_imputed, y)

# Build a ranked summary table
scores = pd.DataFrame({
  &quot;Feature&quot;: num_cols,
  &quot;F_score&quot;: skb.scores_,
  &quot;p_value&quot;: skb.pvalues_
}).sort_values(&quot;F_score&quot;, ascending=False).reset_index(drop=True)
print(scores.to_string(index=False))

# Visualize top 15 features
top = scores.head(15)
plt.figure(figsize=(8, 5))
plt.barh(top[&quot;Feature&quot;][::-1], top[&quot;F_score&quot;][::-1])
plt.xlabel(&quot;F-score (ANOVA)&quot;)
plt.title(&quot;Top 15 Features by Univariate F-score&quot;)
plt.tight_layout()
plt.show()


# Output:
#                        Feature    F_score      p_value
#                      int_rate 436.702249 4.837642e-95
#                inq_last_6mths  94.312329 3.342690e-22
#                   tot_cur_bal  38.503240 5.672972e-10
#              total_rev_hi_lim  24.233665 8.662882e-07
#                total_rev_hi_lim  24.233665 8.662882e-07
# mths_since_last_record_missing  23.277386 1.422172e-06
#                    annual_inc  23.141278 1.526235e-06
```

```
#                    revol_util  21.172802 4.246079e-06
#                   installment  20.043251 7.651060e-06
#        mths_since_last_record  19.848291 8.470742e-06
#                          term  14.982716 1.091591e-04
#                       pub_rec  12.940475 3.230348e-04
#                   delinq_2yrs   6.805131 9.102538e-03
#                     revol_bal   5.893939 1.520981e-02
#                acc_now_delinq   5.085779 2.414339e-02
#        mths_since_last_delinq   4.221569 3.993799e-02
#              emp_length_years   4.164755 4.129841e-02
#                     loan_amnt   4.058954 4.396321e-02
# mths_since_last_delinq_missing   3.668917 5.546341e-02
#                           dti   2.329029 1.270113e-01
#                     total_acc   0.436038 5.090553e-01
#                  tot_coll_amt   0.380460 5.373704e-01
#                      open_acc   0.121032 7.279244e-01
```

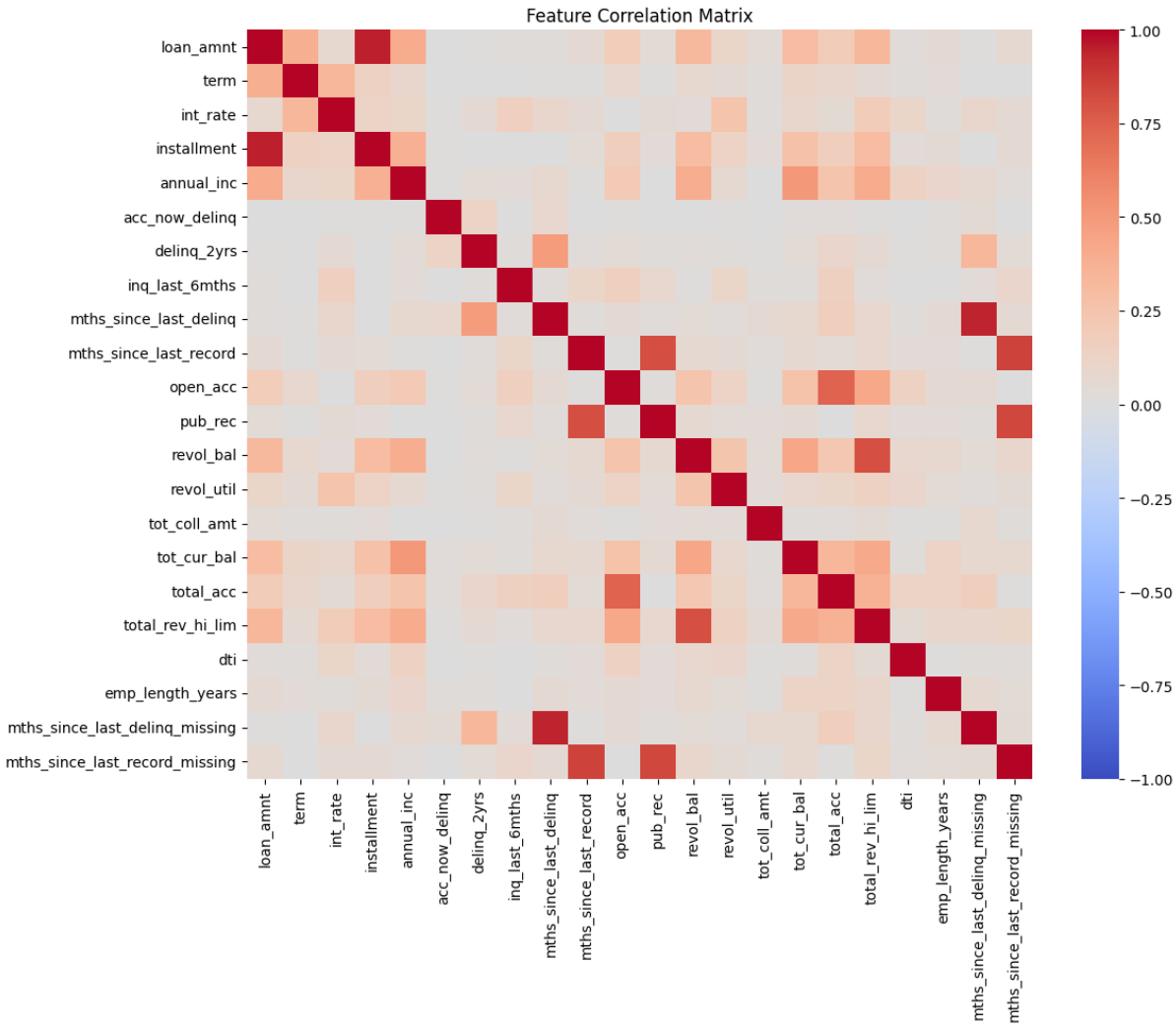


The F-scores reveal which features have the strongest *individual* relationships with the label. Features like *int_rate*, *dti*, and *grade*-related variables tend to score highly because they directly reflect the risk profile of the loan. However, remember that these are bivariate tests—they do not account for interactions or redundancy between features. A feature with a modest F-score may become important when combined with others, and two high-scoring features may be redundant.

High Correlation Filter

When two features are very highly correlated with each other, they carry nearly the same information. In predictive modeling, keeping both is usually harmless. But if you want to simplify the model or if you are working in a causal framework, dropping one of a highly correlated pair is useful. Let's examine the correlation structure of our numeric features.

```
import seaborn as sns

corr = X_num_imputed.corr().abs()

# Find pairs with correlation above 0.80
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))

high_corr_pairs = [
  (col, upper[col].idxmax(), upper[col].max())

  for col in upper.columns if upper[col].max() &gt; 0.80
]

print(&quot;Highly correlated pairs (|r| &gt; 0.80):&quot;)

for f1, f2, r in sorted(high_corr_pairs, key=lambda x: -x[2]):
    print(f&quot;  {f1} &lt;--&gt; {f2}: r = {r:.3f}&quot;)

# Heatmap of correlations
plt.figure(figsize=(12, 10))
sns.heatmap(corr, annot=False, cmap=&quot;coolwarm&quot;, vmin=-1, vmax=1)
plt.title(&quot;Feature Correlation Matrix&quot;)
plt.tight_layout()
plt.show()


# Output:
# Highly correlated pairs (|r| &gt; 0.80):
# installment &lt;--&gt; loan_amnt: r = 0.946
# mths_since_last_delinq_missing &lt;--&gt; mths_since_last_delinq: r = 0.939
# mths_since_last_record_missing &lt;--&gt; mths_since_last_record: r = 0.858
# pub_rec &lt;--&gt; mths_since_last_record: r = 0.819
# total_rev_hi_lim &lt;--&gt; revol_bal: r = 0.808
```



Highly correlated pairs (such as *loan_amnt* and *installment*, which are mechanically linked) are candidates for removal. In a predictive project, you would test whether dropping one improves cross-validated performance. In a causal project, you would drop one to reduce multicollinearity (using VIF, covered in a later section of this chapter).

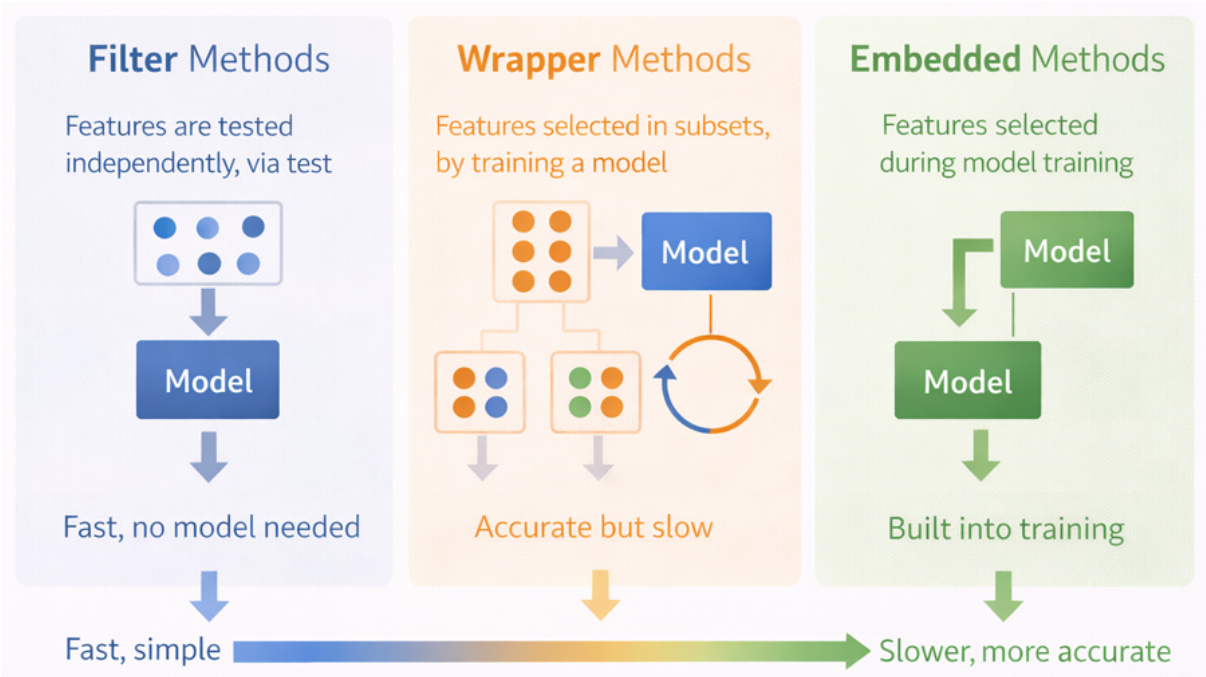
Checkpoint

Filter methods are fast and require no trained model. They are best used as an initial screening step to remove obviously weak or redundant features before applying more computationally expensive techniques. Think of them as the

“quick pass” that narrows 100 features down to 30—then let model-based methods do the fine-tuning.

16.4Wrapper Methods (Model-Based Selection)

Wrapper methods evaluate feature subsets by training a model and measuring its performance. Unlike filters, they capture feature interactions because the model considers all selected features simultaneously. The trade-off is computational cost: wrapper methods must train many models to search the feature space.



Computational Cost

Wrapper methods can be expensive. For p features with k-fold cross-validation, backward elimination requires roughly $p \times k$ model fits per step, and there can be up to p steps. With 30 features and 5-fold CV, that is up to

750 model fits. For large feature sets, start with filter methods to reduce *p* before applying wrappers.

Recursive Feature Elimination with Cross-Validation (RFECV)

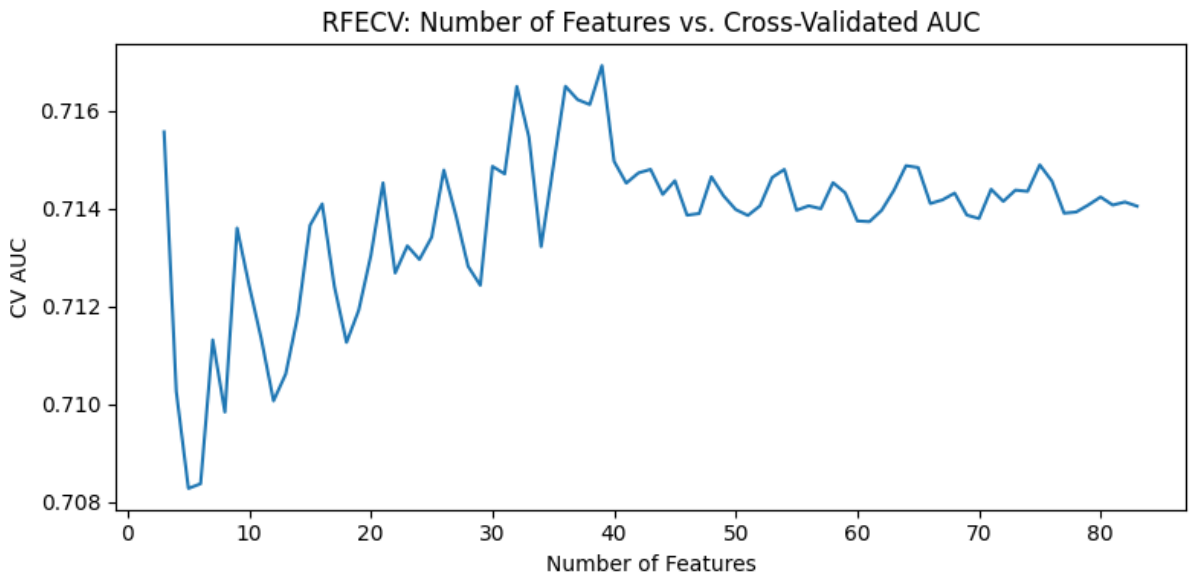
RFECV is one of the most practical wrapper methods in scikit-learn. It works by recursively removing the least important feature (based on the model's internal importance, such as coefficients or tree importance), then cross-validating the remaining set. It continues until it finds the number of features that maximizes cross-validated performance.

Let's apply RFECV using a gradient boosting classifier. We first need to preprocess the data (impute and encode) so that the model can handle it.

```
from sklearn.feature_selection import RFECV
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

# --- Preprocess: impute + encode so RFECV gets a clean matrix ---
numeric_pipe = Pipeline([
  (&quot;impute&quot;, SimpleImputer(strategy=&quot;median&quot;)),
  (&quot;scale&quot;, StandardScaler())
])

categorical_pipe = Pipeline([
  (&quot;impute&quot;, SimpleImputer(strategy=&quot;most_frequent&quot;)),
  (&quot;onehot&quot;, OneHotEncoder(handle_unknown=&quot;ignore&quot;,
sparse_output=False))
])

preprocessor = ColumnTransformer([
  (&quot;num&quot;, numeric_pipe, num_cols),
  (&quot;cat&quot;, categorical_pipe, cat_cols)
])

X_processed = preprocessor.fit_transform(X)

feature_names = (num_cols +
                 preprocessor.named_transformers_[&quot;cat&quot;]
                 .named_steps[&quot;onehot&quot;]
                 .get_feature_names_out(cat_cols).tolist())
```

```
# --- Run RFECV ---
gbc = GradientBoostingClassifier(n_estimators=50, max_depth=3, random_state=27)
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=27)

rfecv = RFECV(estimator=gbc, step=1, cv=cv, scoring=&quot;roc_auc&quot;,
               min_features_to_select=3, n_jobs=-1)

rfecv.fit(X_processed, y)
print(f&quot;Optimal number of features: {rfecv.n_features_}&quot;)
print(f&quot;Best CV AUC: {rfecv.cv_results_['mean_test_score'].max():.4f}&quot;)

# Plot number of features vs CV score
plt.figure(figsize=(8, 4))

plt.plot(range(3, len(rfecv.cv_results_[&quot;mean_test_score&quot;]) + 3),
         rfecv.cv_results_[&quot;mean_test_score&quot;])

plt.xlabel(&quot;Number of Features&quot;)
plt.ylabel(&quot;CV AUC&quot;)
plt.title(&quot;RFECV: Number of Features vs. Cross-Validated AUC&quot;)
plt.tight_layout()
plt.show()

# Show selected features
selected = [f for f, s in zip(feature_names, rfecv.support_) if s]
print(f&quot;\nSelected features ({len(selected)}):&quot;)

for f in selected:
  print(f&quot;  {f}&quot;)


# Output:
# Optimal number of features: 39
# Best CV AUC: 0.7169
#
# Selected features (39):
# loan_amnt
# int_rate
# installment
# annual_inc
# acc_now_delinq
# delinq_2yrs
# inq_last_6mths
# mths_since_last_record
# open_acc
# pub_rec
# revol_bal
# revol_util
# tot_coll_amt
# tot_cur_bal
# total_acc
# total_rev_hi_lim
# dti
# emp_length_years
# home_ownership_RENT
# verification_status_Not Verified
# verification_status_Source Verified
# verification_status_Verified
# grade_C
# grade_D
# grade_E
# grade_F
```

```
# grade_G
# sub_grade_B1
# sub_grade_C2
# sub_grade_C3
# sub_grade_D5
# sub_grade_E1
# sub_grade_E3
# sub_grade_E4
# sub_grade_E5
# sub_grade_F1
# sub_grade_F2
# sub_grade_F4
# sub_grade_F5
```



The RFECV results illustrate a classic feature-selection pattern: rapid early improvement followed by a long performance plateau. As the number of features increases from a very small subset, cross-validated AUC rises quickly, indicating that the model is gaining meaningful predictive signal from the most informative variables. However, after roughly 15–20 features, the curve begins to flatten and performance improvements become incremental.

The peak cross-validated AUC occurs at approximately 39 features, where performance reaches its maximum value of about 0.717. Beyond this point, adding additional features provides little to no improvement and occasionally

produces slight declines in performance. This behavior reflects the tradeoff between information and noise: once the most predictive features are included, additional variables tend to contribute redundancy or noise rather than new signal.

Importantly, the curve remains relatively stable after the plateau, suggesting that the model is fairly robust to moderate changes in feature count. In practice, this means that a slightly smaller feature set (for example, 25–35 features) might perform nearly as well while producing a simpler and more interpretable model. RFECV selects 39 features because that point yields the highest cross-validated AUC, but the broader pattern shows diminishing returns after the initial core set of predictive variables is included.

This visual pattern is the signature of effective wrapper-based feature selection. Early features contribute substantial predictive power, while later additions yield progressively smaller gains. RFECV formalizes this process by identifying the feature subset that maximizes out-of-sample performance, helping us balance model complexity against generalization performance.

SequentialFeatureSelector

Scikit-learn also provides SequentialFeatureSelector, which performs either forward selection (start with no features, add the best one at each step) or backward elimination (start with all features, remove the worst one at each step). Unlike RFECV, it uses cross-validated *model performance* rather than internal feature importance to decide which feature to add or remove.

```
from sklearn.feature_selection import SequentialFeatureSelector

# Forward selection: start with 0 features, add the best one each step
sfs = SequentialFeatureSelector(
```

```
    estimator=gbc,
    n_features_to_select=10,
    direction=&quot;forward&quot;,
    scoring=&quot;roc_auc&quot;,
    cv=cv,
    n_jobs=-1
)

sfs.fit(X_processed, y)
sfs_selected = [f for f, s in zip(feature_names, sfs.get_support()) if s]

print(f&quot;Forward selection chose {len(sfs_selected)} features:&quot;)

for f in sfs_selected:
    print(f&quot;  {f}&quot;)


# Output:
# Forward selection chose 10 features:
#  int_rate
#  inq_last_6mths
#  tot_cur_bal
#  total_rev_hi_lim
#  mths_since_last_record_missing
#  annual_inc
#  revol_util
#  installment
#  mths_since_last_record
#  term
#  pub_rec
#  revol_bal
```

Forward selection is useful when you want a specific number of features or when backward elimination is too slow (because you have many features). However, forward selection is greedier—once a feature is added, it is never reconsidered. RFECV is generally preferred when computationally feasible because it allows features to be reconsidered at each step.

16.5Embedded Methods (Selection During Training)

Embedded methods perform feature selection as part of the model training process itself. They are more efficient than wrapper methods because they do not require training separate models for each feature subset. The two most common embedded approaches are *regularization* (which penalizes model complexity) and *tree-based importance* (which measures how much each feature contributes to splits).

L1 Regularization (Lasso)

L1 regularization (Lasso) adds a penalty proportional to the absolute value of model coefficients. As the penalty increases, it drives the coefficients of weak features to exactly zero—effectively removing them from the model. For classification, we use *LogisticRegression* with *penalty='elasticnet'* and *l1_ratio=1.0* to apply pure L1 regularization (the *saga* solver supports all *l1_ratio* values). During the scikit-learn transition period, *penalty='elasticnet'* is required for *l1_ratio* to take effect; it can be removed once scikit-learn 1.10 is released. The strength of the penalty is controlled by the *C* parameter: smaller C means stronger regularization and fewer features retained.

Scikit-learn's SelectFromModel wraps any estimator that has a *coef_* or *feature_importances_* attribute and selects features whose importance exceeds a threshold.

```
from sklearn.feature_selection import SelectFromModel
from sklearn.linear_model import LogisticRegression

# L1 logistic regression with moderate regularization
# l1_ratio=1.0 applies pure L1 (lasso); saga solver supports all l1_ratio values
# penalty='elasticnet' needed for sklearn &lt;1.10; remove once sklearn ≥1.10
lasso_lr = LogisticRegression(penalty=&quot;elasticnet&quot;, l1_ratio=1.0, C=0.1,
                              solver=&quot;saga&quot;, max_iter=5000,
random_state=27)

lasso_lr.fit(X_processed, y)

# Use SelectFromModel to identify non-zero coefficients
sfm_lasso = SelectFromModel(lasso_lr, prefit=True)
lasso_selected = [f for f, s in zip(feature_names, sfm_lasso.get_support()) if s]

print(f&quot;Lasso selected {len(lasso_selected)} features (C=0.1):&quot;)

for f in lasso_selected:
  coef = lasso_lr.coef_[0][feature_names.index(f)]
  print(f&quot;  {f}: coef = {coef:.4f}&quot;)


# Output:
```

```
# Lasso selected 15 features (C=0.1):
# term: coef = 0.0478
# int_rate: coef = -0.3576
# installment: coef = -0.1402
# annual_inc: coef = 0.1316
# acc_now_delinq: coef = -0.0434
# delinq_2yrs: coef = -0.0366
# inq_last_6mths: coef = -0.1824
# revol_util: coef = -0.0432
# tot_cur_bal: coef = 0.1704
# mths_since_last_record_missing: coef = 0.0866
# home_ownership_RENT: coef = -0.1000
# verification_status_Not Verified: coef = 0.0819
# grade_A: coef = 0.8166
# grade_B: coef = 0.4373
# sub_grade_B2: coef = 0.0676
```

The Lasso approach is particularly elegant because the regularization penalty does double duty: it prevents overfitting *and* performs feature selection simultaneously. Features with zero coefficients are genuinely uninformative given the other features in the model. Adjusting C lets you control how aggressive the selection is—smaller C keeps fewer features.

Why L1 Selects Features but L2 Does Not

It is worth contrasting L1 (Lasso) with *L2 (Ridge)* regularization, because the difference explains why L1 is an embedded feature selection method while L2 is not. L2 regularization adds a penalty proportional to the *squared* value of each coefficient. This penalty shrinks all coefficients toward zero, but it never drives any coefficient to *exactly* zero—every feature remains in the model with a small, non-zero weight. L1, by contrast, adds a penalty proportional to the *absolute* value of each coefficient. The geometry of this constraint—a diamond-shaped region in coefficient space—creates corners at the coordinate axes where one or more coefficients can be exactly zero. When the optimal solution falls on a corner, that feature is eliminated entirely.

We can demonstrate this directly by training both L1 and L2 logistic regression models on the same data with the same regularization strength (C=0.1):

```
# Compare L1 (Lasso) vs L2 (Ridge) regularization
ridge_lr = LogisticRegression(penalty=&quot;l2&quot;, C=0.1,
                              solver=&quot;lbfgs&quot;, max_iter=5000,
random_state=27)

ridge_lr.fit(X_processed, y)
lasso_zero = int(np.sum(lasso_lr.coef_[0] == 0))
ridge_zero = int(np.sum(ridge_lr.coef_[0] == 0))

print(f&quot;L1 (Lasso): {lasso_zero} of {len(feature_names)} coefficients driven to
zero&quot;)
print(f&quot;L2 (Ridge): {ridge_zero} of {len(feature_names)} coefficients driven to
zero&quot;)


# Output:
# L1 (Lasso): 59 of 83 coefficients driven to zero
# L2 (Ridge): 0 of 83 coefficients driven to zero
```

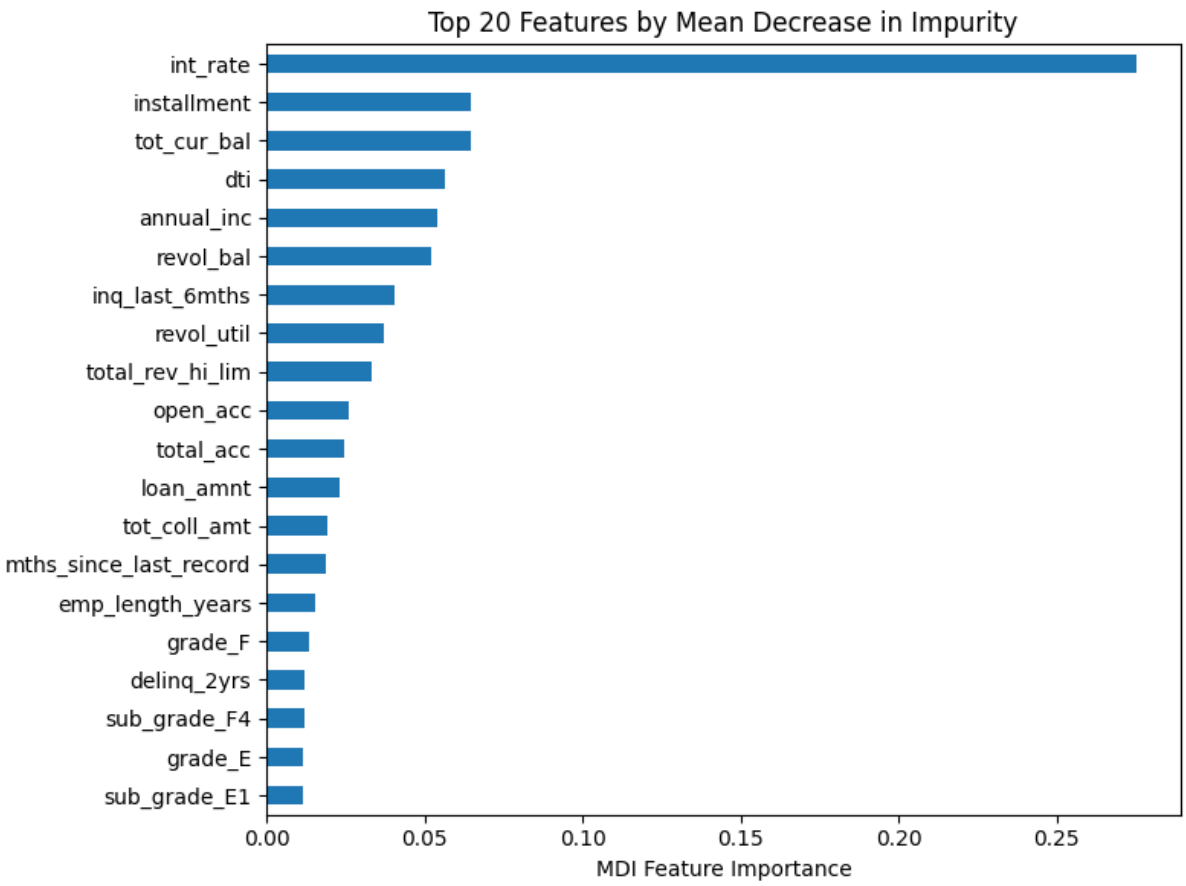
The result is stark: with the same regularization strength, L1 drives 59 of 83 coefficients to exactly zero, while L2 keeps every single coefficient non-zero. This is why L1 is classified as an embedded feature selection method—it performs selection as a natural consequence of its penalty structure. L2 regularization is valuable for preventing overfitting, but it is *not* a feature selection method because it retains all features in the final model.

Tree-Based Feature Importance (MDI)

Tree-based models like random forests and gradient boosting compute a *mean decrease in impurity* (MDI) for each feature during training. MDI measures how much each feature contributes to reducing the splitting criterion (Gini impurity for classification, MSE for regression) across all trees in the ensemble. Features used in early, high-impact splits get higher importance scores.

We can use SelectFromModel with a tree-based estimator to keep only features above the mean importance threshold.

```
from sklearn.ensemble import GradientBoostingClassifier

# Train a gradient boosting model
gbc_full = GradientBoostingClassifier(n_estimators=200, max_depth=3,
random_state=27)
gbc_full.fit(X_processed, y)

# Extract and plot MDI importance
mdi = pd.Series(gbc_full.feature_importances_, index=feature_names)
mdi = mdi.sort_values(ascending=False)
plt.figure(figsize=(8, 6))
mdi.head(20).sort_values().plot(kind=&quot;barh&quot;)
plt.xlabel(&quot;MDI Feature Importance&quot;)
plt.title(&quot;Top 20 Features by Mean Decrease in Impurity&quot;)
plt.tight_layout()
plt.show()

# SelectFromModel with tree importance
sfm_tree = SelectFromModel(gbc_full, prefit=True, threshold=&quot;mean&quot;)
tree_selected = [f for f, s in zip(feature_names, sfm_tree.get_support()) if s]

print(f&quot;\nSelectFromModel kept {len(tree_selected)} features (above mean
importance):&quot;)

for f in tree_selected:
  print(f&quot;  {f}: MDI = {mdi[f]:.4f}&quot;)


# Output:
# SelectFromModel kept 18 features (above mean importance):
# loan_amnt: MDI = 0.0232
# int_rate: MDI = 0.2755
# installment: MDI = 0.0648
# annual_inc: MDI = 0.0540
# delinq_2yrs: MDI = 0.0122
# inq_last_6mths: MDI = 0.0407
# mths_since_last_record: MDI = 0.0190
# open_acc: MDI = 0.0259
# revol_bal: MDI = 0.0520
# revol_util: MDI = 0.0374
# tot_coll_amt: MDI = 0.0192
# tot_cur_bal: MDI = 0.0645
# total_acc: MDI = 0.0248
# total_rev_hi_lim: MDI = 0.0335
# dti: MDI = 0.0566
# emp_length_years: MDI = 0.0156
# grade_F: MDI = 0.0137
# sub_grade_F4: MDI = 0.0121
```



Caveats of MDI Importance

MDI is fast and intuitive, but it has well-documented limitations:

- **Bias toward high-cardinality features:** Features with many unique values (such as continuous numerics) can appear more important than features with fewer values (such as binary indicators), even when the low-cardinality feature is genuinely more predictive.
- **Correlated features split importance:** When two features are correlated, importance is divided between them. Each may appear less important individually than it would if the other were removed.
- **Importance ≠ causality:** A high MDI score means the feature is useful for prediction, not that it causes the outcome.

Despite these caveats, tree-based models have a notable advantage: they are naturally robust to irrelevant features. Unlike linear models, which assign a coefficient to every feature regardless of usefulness, trees only select features that produce meaningful splits. An irrelevant feature will rarely be chosen for a split because it does not reduce impurity. This means that in many practical settings—particularly when the feature count is moderate relative to the sample size—adding irrelevant features does not significantly hurt tree-based model performance. Feature selection for tree models is therefore more about *simplifying the model* and *improving interpretability* than about improving predictive accuracy.

These limitations motivate permutation feature importance, which we cover next.

16.6Permutation Feature Importance

Permutation feature importance (PFI) A model-agnostic technique that measures feature importance by randomly shuffling each feature's values and observing how much model performance degrades. Larger degradation indicates higher importance. is a model-agnostic technique for measuring feature importance. Unlike MDI (which is computed during training and specific to tree models) or coefficients (specific to linear models), PFI works with *any* fitted model—trees, linear models, neural networks, SVMs, or anything else.

How PFI Works

The idea is simple and elegant:

1. Train a model and record its baseline performance on a held-out set.

2. For each feature, randomly shuffle (permute) that feature's values while keeping all other features intact.
3. Re-score the model on the shuffled data and record the performance drop.
4. Repeat the shuffle multiple times (e.g., 10 repeats) and average the drops to get a stable estimate.

Features that cause the largest performance drop when shuffled are the most important—the model relied on them heavily. Features whose shuffling causes little or no drop are unimportant and are candidates for removal.

```
from sklearn.inspection import permutation_importance

# Split for proper evaluation (PFI should be computed on held-out data)
X_train, X_test, y_train, y_test = train_test_split(
    X_processed, y, test_size=0.20, random_state=27, stratify=y
)

# Train model on training set
gbc_pfi = GradientBoostingClassifier(n_estimators=200, max_depth=3, random_state=27)
gbc_pfi.fit(X_train, y_train)

# Compute PFI on test set
pfi_result = permutation_importance(
    gbc_pfi, X_test, y_test,
    n_repeats=10, scoring=&quot;roc_auc&quot;, random_state=27, n_jobs=-1
)

pfi = pd.DataFrame({
    &quot;Feature&quot;: feature_names,
    &quot;PFI_mean&quot;: pfi_result.importances_mean,
    &quot;PFI_std&quot;: pfi_result.importances_std
}).sort_values(&quot;PFI_mean&quot;, ascending=False)

# Show top features
print(pfi.head(15).to_string(index=False))

# Plot PFI
top_pfi = pfi.head(15).sort_values(&quot;PFI_mean&quot;)
plt.figure(figsize=(8, 5))
plt.barh(top_pfi[&quot;Feature&quot;], top_pfi[&quot;PFI_mean&quot;],
xerr=top_pfi[&quot;PFI_std&quot;])
plt.xlabel(&quot;Permutation Importance (decrease in AUC)&quot;)
plt.title(&quot;Top 15 Features by Permutation Importance&quot;)
plt.tight_layout()
plt.show()
```

```
# Output:
#               Feature  PFI_mean  PFI_std
#              int_rate  0.123016 0.021873
#             loan_amnt  0.037719 0.007567
#          installment  0.031365 0.004450
#            revol_bal  0.011200 0.003797
#          tot_cur_bal  0.010050 0.005257
#         sub_grade_C4  0.008744 0.002915
#       inq_last_6mths  0.007778 0.002921
#                  dti  0.007489 0.004532
#              grade_E  0.005728 0.002467
#             open_acc  0.005607 0.003741
#           annual_inc  0.004440 0.003371
#     total_rev_hi_lim  0.003671 0.003912
#            total_acc  0.003567 0.002565
#          delinq_2yrs  0.002023 0.000881
#              grade_F  0.001816 0.001099
```

image

The permutation feature importance results provide a model-agnostic view of which variables the trained gradient boosting model truly depends on for predictive performance. The bar chart shows the average decrease in AUC when each feature is randomly shuffled on the held-out test set. A clear hierarchy emerges. The interest rate (*int_rate*) dominates all other variables, producing by far the largest drop in AUC (approximately 0.12) when permuted, indicating that the model relies heavily on this feature to distinguish between outcomes. A second tier of importance includes loan amount (*loan_amnt*) and installment size (*installment*), followed by balance and credit history variables such as revolving balance, total current balance, and recent inquiries. Many remaining variables have very small importance values with error bars overlapping zero, suggesting that permuting them does not materially degrade predictive performance.

Several practical implications follow. First, the model's predictive signal is concentrated in a relatively small set of financial and credit-risk variables, meaning that a simpler feature set may achieve nearly the same performance as the full dataset. Second, features with near-zero or highly unstable

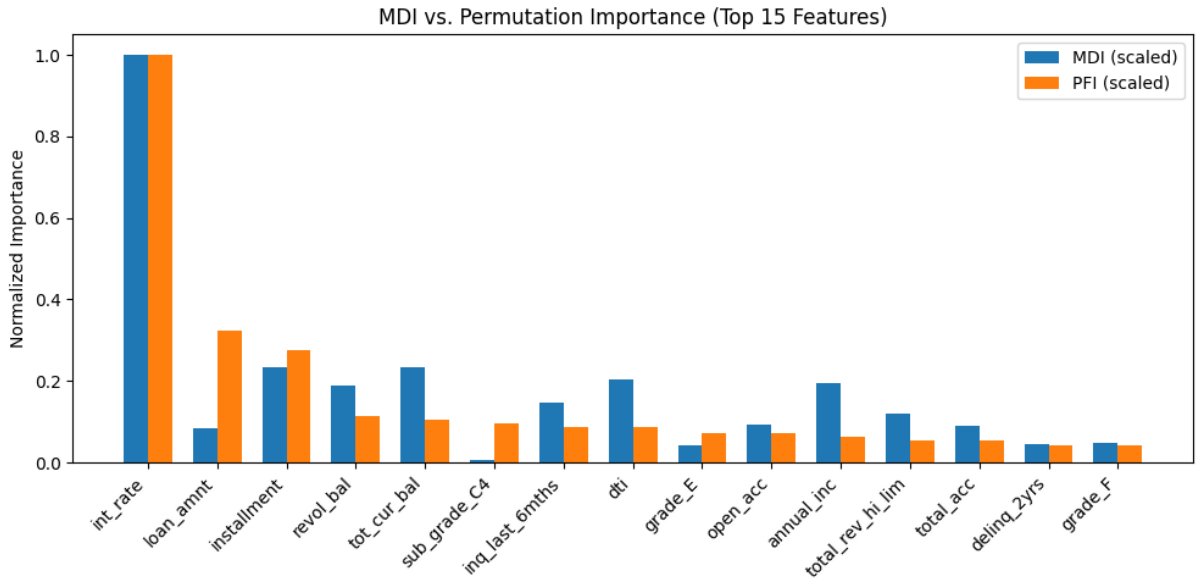
permutation importance are candidates for removal, especially if they add data collection cost, complexity, or risk of leakage. Third, the large gap between the top feature and all others highlights potential model sensitivity: if interest rate data were unavailable or measured poorly, performance would likely degrade substantially. From a decision-making perspective, these results help prioritize data quality efforts, guide feature pruning, and clarify which variables carry the greatest business and predictive value.

Comparing MDI and PFI

MDI and PFI often agree on the most important features but can disagree on less important ones. Let’s compare them side by side.

```
from sklearn.preprocessing import MinMaxScaler

# Merge MDI and PFI into one comparison table
comparison = pd.DataFrame({
    &quot;Feature&quot;: feature_names,
    &quot;MDI&quot;: gbc_full.feature_importances_,
    &quot;PFI&quot;: pfi_result.importances_mean
})

# Normalize both to 0-1 range for visual comparison
comparison[&quot;MDI_scaled&quot;] =
MinMaxScaler().fit_transform(comparison[[&quot;MDI&quot;]])
comparison[&quot;PFI_scaled&quot;] =
MinMaxScaler().fit_transform(comparison[[&quot;PFI&quot;]])
comparison = comparison.sort_values(&quot;PFI_scaled&quot;, ascending=False)

# Plot side-by-side
top_compare = comparison.head(15)
fig, ax = plt.subplots(figsize=(10, 5))
x = range(len(top_compare))
width = 0.35
ax.bar([i - width/2 for i in x], top_compare[&quot;MDI_scaled&quot;], width,
label=&quot;MDI (scaled)&quot;)
ax.bar([i + width/2 for i in x], top_compare[&quot;PFI_scaled&quot;], width,
label=&quot;PFI (scaled)&quot;)
ax.set_xticks(x)
ax.set_xticklabels(top_compare[&quot;Feature&quot;], rotation=45,
ha=&quot;right&quot;)
ax.set_ylabel(&quot;Normalized Importance&quot;)
ax.set_title(&quot;MDI vs. Permutation Importance (Top 15 Features)&quot;)
ax.legend()
plt.tight_layout()
```

```
plt.show()
```



Where MDI and PFI disagree, PFI is generally more trustworthy because it is computed on held-out data and does not suffer from the high-cardinality bias that affects MDI. However, PFI has its own caveat: when two features are highly correlated, shuffling one has little effect because the model can still use the other. This can make both features appear less important than they truly are.

Two additional properties of PFI are worth noting. First, PFI scales well to datasets of any size—more observations actually produce more stable importance estimates because the shuffled performance measurements have less variance. PFI does not become unreliable on large datasets; if anything, larger samples make the importance estimates more precise. Second, neither MDI nor PFI inherently adjusts for class imbalance. Both methods handle imbalanced datasets through the *scoring metric* you choose: using ROC AUC or F1 rather than raw accuracy ensures that importance reflects genuine predictive contribution regardless of class proportions. The importance

method itself is agnostic to class balance—it is the scoring function that determines whether minority-class performance is properly weighted.

The side-by-side comparison highlights both agreement and meaningful differences between mean decrease in impurity (MDI) and permutation feature importance (PFI). At the top of the ranking, the two methods strongly agree: *int_rate* is clearly the dominant predictor by a wide margin under both metrics, reinforcing confidence that this variable carries the most predictive signal. Several additional variables—including loan amount, installment size, and key credit history measures—also appear consistently important across both methods, suggesting that the model's core predictive structure is stable and not an artifact of any single importance calculation.

However, noticeable differences emerge in the middle of the ranking. MDI assigns relatively higher importance to variables such as total current balance, annual income, and total revolving credit limits, while PFI places comparatively more emphasis on loan amount and certain credit-grade indicators. These differences arise because MDI measures how often a feature improves splits within the training trees, whereas PFI measures the actual drop in predictive performance on held-out data when the feature's information is disrupted. When the two metrics diverge, PFI generally provides the more reliable estimate of real-world predictive contribution because it reflects out-of-sample impact rather than internal model mechanics.

From a decision perspective, the strong agreement on the top features indicates that model performance depends heavily on a small, consistent core set of variables. Features with moderate disagreement between MDI and PFI should be interpreted cautiously: they may contribute to tree construction without materially improving generalization, or they may share predictive

information with correlated variables. For feature selection and model simplification, this comparison suggests prioritizing features that rank highly under both metrics while scrutinizing those that appear important only under MDI. Using both views together provides a more nuanced understanding of which variables truly drive predictive performance and which merely support model structure.

Checkpoint
Permutation feature importance is the most advanced feature importance metric covered in this book. It works with any model, uses held-out data, and provides confidence intervals via multiple repeats. For post-model feature analysis, PFI should be your default choice. More advanced explanation tools like SHAP and LIME, which provide per-prediction explanations rather than global importance rankings, are covered in the next book.

16.7Causal Feature Selection

Everything we have covered so far—filters, wrappers, embedded methods, and PFI—serves the *predictive* paradigm. These methods optimize for model performance. But when your goal is to understand *why* something happens rather than *what will* happen, you need a fundamentally different approach.

Causal feature selection is guided by theory and diagnostics rather than cross-validated accuracy. The central question is not “does this feature improve predictions?” but “does this feature allow me to trust my coefficient estimates?”

Variance Inflation Factor (VIF)

The *variance inflation factor* (VIF) quantifies how much a feature’s coefficient variance is inflated by its correlation with all other features in the

model. A VIF of 1 means no multicollinearity. A VIF of 5 means the coefficient's variance is 5 times what it would be if the feature were uncorrelated with the others—making the coefficient estimate 5 times less precise.

Let's compute VIF for the numeric features in our Lending Club dataset. Imagine we are building an explanatory model to answer: "What borrower characteristics are most strongly associated with loan default?"

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

# VIF requires complete cases and numeric data only
X_vif = X_num_imputed.copy()

# Add constant for VIF calculation
X_vif.insert(0, &quot;const&quot;, 1)

vif_data = pd.DataFrame({
    &quot;Feature&quot;: X_vif.columns[1:],  # Skip the constant
    &quot;VIF&quot;: [variance_inflation_factor(X_vif.values, i)
             for i in range(1, X_vif.shape[1])]
}).sort_values(&quot;VIF&quot;, ascending=False)
print(vif_data.to_string(index=False))


# Output:
#                        Feature       VIF
#                          loan_amnt 47.583770
#                        installment 40.959956
#             mths_since_last_delinq 11.885828
#     mths_since_last_delinq_missing 10.295649
#                               term  5.585071
#                   total_rev_hi_lim  5.493045
#     mths_since_last_record_missing  4.947173
#                          revol_bal  4.891111
#             mths_since_last_record  4.408138
#                            pub_rec  3.909427
#                          total_acc  2.515391
#                           open_acc  2.491837
#                           int_rate  2.335982
#                         revol_util  1.739049
#                         annual_inc  1.627667
#                        delinq_2yrs  1.584069
#                        tot_cur_bal  1.582454
#                      inq_last_6mths  1.099009
#                                 dti  1.093299
#                   emp_length_years  1.038247
#                      acc_now_delinq  1.020635
#                        tot_coll_amt  1.015807
```

You will likely see features like *loan_amnt* and *installment* with very high VIF scores. This makes sense: the monthly installment is mechanically determined by the loan amount, interest rate, and term. In a predictive model, keeping both is fine. In a causal model, keeping both makes the coefficients for each one unreliable because the model cannot separate their individual effects.

The VIF results illustrate a central tension in explanatory modeling: many of the features that improve prediction can undermine interpretation. Extremely high VIF values for *loan_amnt* and *installment* indicate severe multicollinearity—these variables move together so closely that the model cannot reliably estimate their independent effects. In a causal setting, this inflates standard errors and makes coefficient estimates unstable, meaning small data changes could lead to large swings in estimated impact. For explanatory analysis, this typically implies choosing one representative variable (for example, loan amount) and removing or redefining the others to preserve interpretability. More moderate VIF values for variables such as *total_rev_hi_lim* and *revol_bal* suggest potential redundancy but not necessarily disqualifying overlap, requiring judgment informed by domain theory. Features with low VIF values—including debt-to-income ratio, recent inquiry counts, and delinquency indicators—provide more stable coefficient estimates and can generally remain in an explanatory model. The key implication is that causal feature selection is less about maximizing predictive accuracy and more about ensuring that remaining variables support credible, interpretable estimates of effect. Removing or consolidating highly collinear predictors improves coefficient stability, strengthens inference, and ultimately produces models that decision-makers can trust when interpreting relationships rather than simply generating predictions.

Iterative VIF Removal

A common practice is to iteratively remove the feature with the highest VIF, recalculate all VIF scores, and repeat until all remaining features are below a threshold (typically 5 or 10). Here is a function that automates this process.

```
def vif_removal(df, threshold=5.0, verbose=True):
  &quot;&quot;&quot;Iteratively remove the feature with the highest VIF until all
are below threshold.&quot;&quot;&quot;
  import pandas as pd
  from statsmodels.stats.outliers_influence import variance_inflation_factor

  X = df.copy()
  dropped = []

  while True:
    X_const = X.copy()
    X_const.insert(0, &quot;const&quot;, 1)

    vifs = pd.Series(
      [variance_inflation_factor(X_const.values, i)
        for i in range(1, X_const.shape[1])],
      index=X.columns
    )

    max_vif = vifs.max()

    if max_vif &lt; threshold:
      break

    worst = vifs.idxmax()
    dropped.append((worst, round(max_vif, 2)))
    X = X.drop(columns=[worst])

    if verbose:
      print(f&quot;Dropped {worst} (VIF = {max_vif:.2f}), {X.shape[1]} features
remain&quot;)

  if verbose:
    print(f&quot;\nAll remaining features have VIF &lt; {threshold}&quot;)
    print(f&quot;Features dropped: {len(dropped)}&quot;)
    print(f&quot;Features remaining: {X.shape[1]}&quot;)

  return X, dropped

X_causal, dropped_features = vif_removal(X_num_imputed, threshold=5.0)


# Output:
# Dropped loan_amnt (VIF = 47.58), 21 features remain
# Dropped mths_since_last_delinq (VIF = 11.89), 20 features remain
# Dropped total_rev_hi_lim (VIF = 5.49), 19 features remain
# All remaining features have VIF &lt; 5.0
# Features dropped: 3
# Features remaining: 19
```

After VIF removal, the remaining features have coefficient estimates that are much more stable and interpretable. You can now fit a logistic regression and interpret each coefficient as an approximate effect of that feature on default probability, holding all other features constant.

The VIF results illustrate a central tension in explanatory modeling: many of the features that improve prediction can undermine interpretation. Extremely high VIF values for *loan_amnt* and *installment* indicate severe multicollinearity—these variables move together so closely that the model cannot reliably estimate their independent effects. In a causal setting, this inflates standard errors and makes coefficient estimates unstable, meaning small data changes could lead to large swings in estimated impact. For explanatory analysis, this typically implies choosing one representative variable (for example, loan amount) and removing or redefining the others to preserve interpretability. More moderate VIF values for variables such as *total_rev_hi_lim* and *revol_bal* suggest potential redundancy but not necessarily disqualifying overlap, requiring judgment informed by domain theory. Features with low VIF values—including debt-to-income ratio, recent inquiry counts, and delinquency indicators—provide more stable coefficient estimates and can generally remain in an explanatory model. The key implication is that causal feature selection is less about maximizing predictive accuracy and more about ensuring that remaining variables support credible, interpretable estimates of effect. Removing or consolidating highly collinear predictors improves coefficient stability, strengthens inference, and ultimately produces models that decision-makers can trust when interpreting relationships rather than simply generating predictions.

Domain-Driven Exclusion

VIF handles statistical redundancy, but it cannot detect *theoretical* problems. Even if a feature has a low VIF, it should be excluded from a causal model if it is:

- **A mediator:** A variable that sits on the causal path between your treatment and outcome. Including it "blocks" the causal effect you are trying to measure.
- **A collider:** A variable caused by both the treatment and the outcome. Including it introduces spurious associations.
- **A proxy for the outcome:** Like our outcome-adjacent payment columns—they are consequences of default, not causes of it.

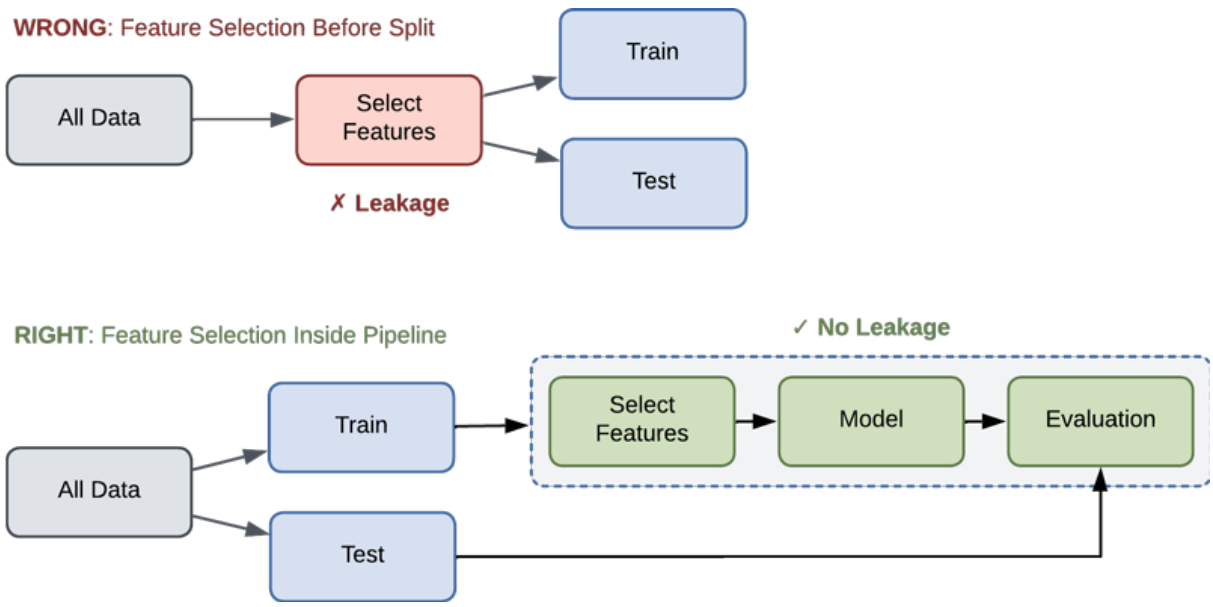
In the Lending Club context, *grade* and *sub_grade* are interesting cases. They are assigned by the lender based on the borrower's risk profile. If you are studying "what borrower characteristics cause defaults," including grade may be a collider—it is influenced by many of the same borrower characteristics (income, credit history) that also predict default. A careful causal analysis would consider whether grade should be included or excluded based on the specific causal question being asked.

This kind of reasoning cannot be automated. It requires domain knowledge and careful thought about the data-generating process. That is why causal feature selection is fundamentally a human activity supported by statistical tools, while predictive feature selection can be largely automated.

16.8Feature Selection Inside Pipelines

One of the most common mistakes in applied machine learning is performing feature selection *before* splitting the data or outside of cross-validation. This causes *data leakage*: the selection step sees information from the test set (or

validation fold), which inflates performance estimates and produces models that appear better than they actually are.



The Wrong Way

Here is an example of what *not* to do:

```
# WRONG: Feature selection on ALL data before cross-validation
from sklearn.model_selection import cross_val_score

skb_leak = SelectKBest(f_classif, k=10)
X_selected = skb_leak.fit_transform(X_num_imputed, y)  # Sees ALL data

scores_leak = cross_val_score(
  GradientBoostingClassifier(n_estimators=100, random_state=27),
  X_selected, y, cv=5, scoring=&quot;roc_auc&quot;
)

print(f&quot;WRONG (leaky): CV AUC = {scores_leak.mean():.4f}&quot;)


# Output: WRONG (leaky): CV AUC = 0.7163
```

The problem becomes clear when we examine the reported result: a cross-validated AUC of 0.7163. At first glance, this appears to be a strong model.

However, the feature selection step was performed using *all* observations before cross-validation began. This means the F-scores used to select the top 10 features were calculated using information from both the training and validation folds. As a result, the model effectively receives a preview of the validation data during feature selection. This produces an optimistically biased performance estimate—the model appears more accurate than it would be on truly unseen data. In practice, a model evaluated this way often performs noticeably worse when deployed or when tested on a genuinely new dataset. The key lesson is that any step that learns from the data—including feature selection—must be confined to the training portion of each fold to preserve the integrity of performance estimates.

The Right Way

The correct approach puts feature selection *inside* the pipeline so that it is re-fitted on each training fold independently:

```
# CORRECT: Feature selection inside the pipeline
from sklearn.pipeline import Pipeline

pipe_correct = Pipeline([
    (&quot;select&quot;, SelectKBest(f_classif, k=10)),
    (&quot;model&quot;, GradientBoostingClassifier(n_estimators=100,
random_state=27))
])

scores_correct = cross_val_score(
    pipe_correct, X_num_imputed, y, cv=5, scoring=&quot;roc_auc&quot;
)

print(f&quot;CORRECT (no leakage): CV AUC = {scores_correct.mean():.4f}&quot;)
print(f&quot;\nDifference: {scores_leak.mean() - scores_correct.mean():.4f}&quot;)
print(&quot;The leaky version appears better, but it is lying to you.&quot;)


# Output:
# CORRECT (no leakage): CV AUC = 0.7173
# Difference: -0.0009
# The leaky version appears better, but it is lying to you.
```

When feature selection is placed inside the pipeline, it is re-estimated separately within each cross-validation training fold, preventing information from leaking into validation data. The resulting cross-validated AUC of 0.7173 reflects a more trustworthy estimate of true out-of-sample performance. Interestingly, the difference between the leaky and non-leaky approaches in this example is small (approximately 0.0009), but that should not be reassuring. Leakage does not always produce large, obvious inflation; in many real projects it introduces subtle bias that accumulates across modeling decisions and leads to overly optimistic conclusions. Even when the numeric difference appears minor, the conceptual error is serious because it breaks the fundamental rule of evaluation: the model must never learn from the data used to judge it. By embedding feature selection inside the pipeline, you ensure that every fold simulates the real deployment scenario in which the model sees only training data when selecting features and fitting parameters. This discipline produces performance estimates you can trust and models that are far more likely to generalize reliably to new data.

Full Pipeline with Preprocessing and Selection

For a complete, production-ready pipeline that handles preprocessing, feature selection, and modeling without leakage, you can nest a ColumnTransformer (for imputation and encoding) with a SelectKBest or SelectFromModel step inside a single Pipeline:

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.feature_selection import SelectFromModel

# Build the full pipeline: preprocess → select → model
full_pipe = Pipeline([
  (&quot;preprocess&quot;, ColumnTransformer([
    (&quot;num&quot;, Pipeline([
      (&quot;impute&quot;, SimpleImputer(strategy=&quot;median&quot;)),
```

```
            (&quot;scale&quot;, StandardScaler())
          ]), num_cols),
          (&quot;cat&quot;, Pipeline([
            (&quot;impute&quot;, SimpleImputer(strategy=&quot;most_frequent&quot;)),
            (&quot;onehot&quot;, OneHotEncoder(handle_unknown=&quot;ignore&quot;,
sparse_output=False))
          ]), cat_cols)
        ])),
        (&quot;select&quot;, SelectFromModel(
          GradientBoostingClassifier(n_estimators=100, max_depth=3, random_state=27),
          threshold=&quot;mean&quot;
        )),
        (&quot;model&quot;, GradientBoostingClassifier(n_estimators=200, max_depth=3,
random_state=27))
      ])

      # Cross-validate the entire pipeline — no leakage possible
      scores_full = cross_val_score(full_pipe, X, y, cv=5, scoring=&quot;roc_auc&quot;)
      print(f&quot;Full pipeline CV AUC: {scores_full.mean():.4f} (+/-
{scores_full.std():.4f})&quot;)


      # Output: Full pipeline CV AUC: 0.7118 (+/- 0.0264)
```

This full pipeline produces a cross-validated AUC of 0.7118 with a standard deviation of approximately 0.0264 across folds, giving a realistic estimate of how the model is likely to perform on unseen data. Notice that this value is slightly lower than some earlier results obtained without full pipeline encapsulation. That difference is expected and desirable: by placing preprocessing and feature selection inside the pipeline, each cross-validation fold simulates a true production scenario in which the model must learn entirely from its training subset. No information from validation folds influences imputation choices, scaling parameters, encoded categories, or selected features. The resulting performance estimate is therefore more conservative but far more trustworthy. The reported standard deviation also highlights an important practical insight: model performance varies across folds because each fold represents a different sample of the underlying population. A stable pipeline should produce not only strong average performance but also reasonably consistent results across folds. In practice, this type of end-to-end pipeline structure is the correct template for deployment-ready modeling workflows, ensuring that every transformation

applied during training will be applied identically to new data in production while preserving the integrity of model evaluation.

Checkpoint
If you take one thing from this section, let it be this: *feature selection must happen inside your cross-validation loop.* If SelectKBest, RFECV, or any other selection method sees test data during fitting, your performance estimates are inflated and your model is less reliable than you think.

16.9Decision Framework

With so many feature selection techniques available, how do you choose? The decision depends on your project goal, the size of your feature set, and the computational budget you have available.

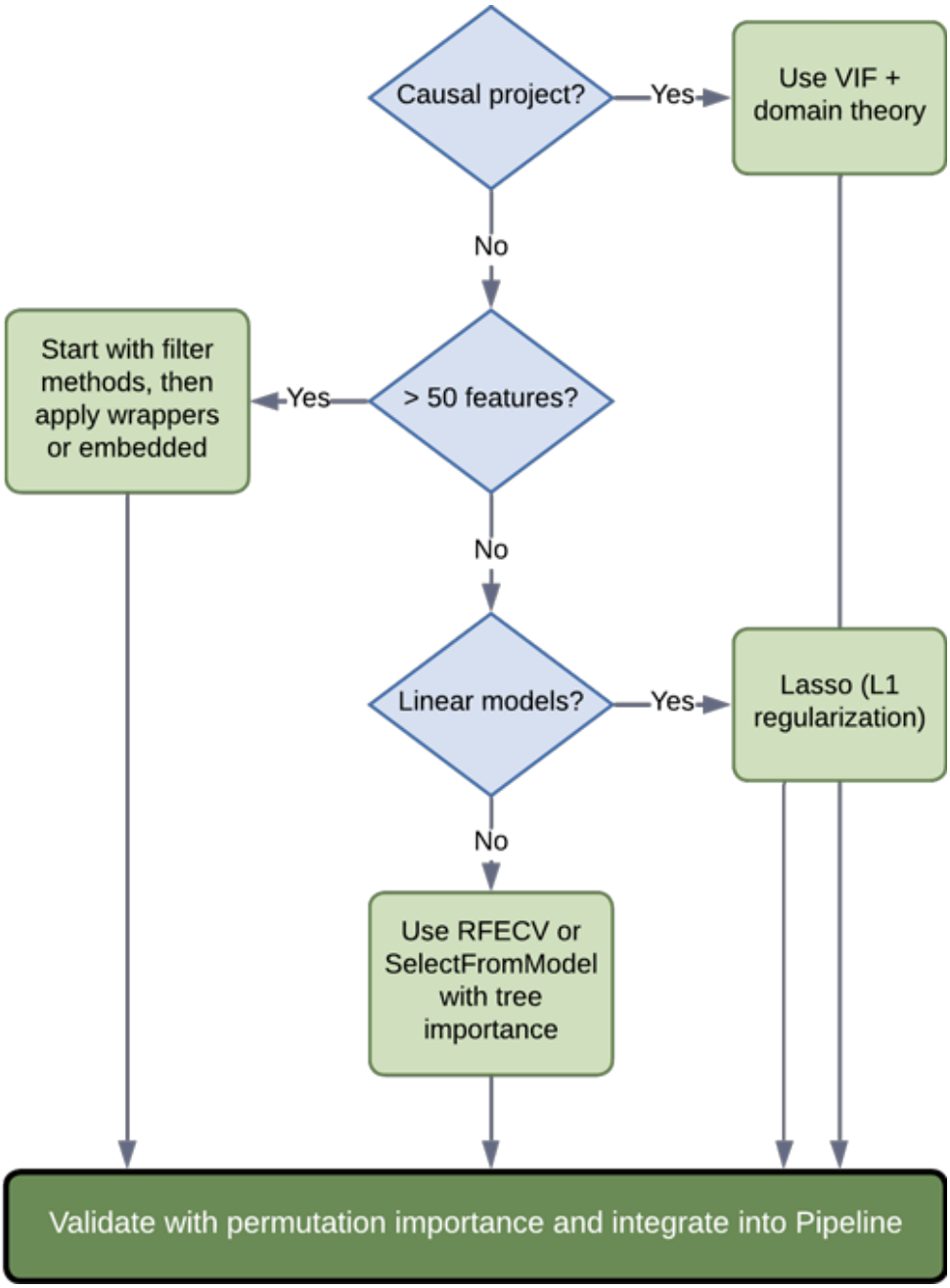
Quick Reference

Table 16.2

Situation	Recommended Method	Why
Causal / explanatory project	VIF + domain theory	Protects coefficient interpretability
Many features (>50), need quick reduction	Filter methods (VarianceThreshold, SelectKBest)	Fast, no model needed, good first pass
Want optimal feature subset, moderate feature count	RFECV	Cross-validated, finds optimal count automatically
Using linear models	L1 regularization (Lasso)	Built-in selection via zero coefficients
Using tree-based models	SelectFromModel with tree importance	Fast, leverages native importance

Want model-agnostic importance ranking	Permutation feature importance	Works with any model, uses held-out data
Production pipeline	Selection inside Pipeline + cross_val_score	Prevents data leakage

Decision Flowchart



Common Mistakes

1. **Using VIF removal for predictive projects:** Multicollinearity does not hurt prediction—only interpretation. Removing correlated features based on VIF in a predictive project can reduce accuracy unnecessarily.

2. **Ignoring multicollinearity in causal projects:** Keeping highly correlated features in an explanatory model produces coefficients that are unreliable and potentially misleading for decision-makers.
3. **Feature selection outside cross-validation:** This is data leakage. The selection step sees test data, inflating performance estimates and producing overconfident models.
4. **Trusting a single importance metric:** MDI, PFI, and coefficients can disagree. When they do, investigate why rather than blindly following one metric. Correlated features, high-cardinality bias, and class imbalance can all cause individual metrics to mislead.

A Note on SHAP and LIME

You may encounter references to SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations) in feature selection discussions. These are powerful explainability tools that can show how much each feature contributes to individual predictions. However, their primary purpose is model *explanation*—helping stakeholders understand why a model made a specific decision—rather than model *building*. SHAP values should not be used to drop features without first validating that removal does not hurt cross-validated performance. These tools are covered in depth in the next book.

For this book, permutation feature importance is the most advanced and trustworthy metric for evaluating which features matter to your model. It shares conceptual DNA with SHAP (both measure what happens when a feature's contribution is disrupted) but is simpler to implement and interpret.

16.10Case Studies

Consider working through these case studies to practice the feature selection skills taught in this chapter. Each case applies different selection methods to a familiar dataset and asks you to compare the results, interpret the differences, and recommend an approach:

Case #1: Customer Churn (Filter vs Embedded Feature Selection)
This case uses the **Customer Churn** dataset to compare **filter methods** (SelectKBest) against **embedded methods** (SelectFromModel) for a binary churn classification problem. Your goal is to apply both approaches, compare the features they select, validate with permutation feature importance, and determine whether feature selection improves model performance.

Dataset attribution: Telecommunications customer churn dataset with demographics, service usage, contract attributes, and a binary churn outcome variable (7,043 rows, 21 columns). See details on Kaggle.com The churn dataset is available in the prior chapter if you need to reload it.

Prediction goal: Predict whether a customer will churn (*Yes* or *No*) using all available features except the target variable.

For reproducibility, use **random_state = 27** everywhere a random seed is accepted.

Tasks

- Load the dataset, convert *TotalCharges* to numeric (use *pd.to_numeric* with *errors='coerce'*), and drop the *customerID* column. Define *X* and *y* (where *y = Churn*). Identify numeric and categorical columns.
- Build a leakage-safe preprocessing pipeline using *ColumnTransformer*: impute and scale numeric features with *SimpleImputer(strategy='median')* and *StandardScaler*; impute and one-

hot encode categorical features with *SimpleImputer(strategy='most_frequent')* and *OneHotEncoder(drop='first')*. Freeze an 80/20 stratified train/test split with *random_state=27*. Fit the preprocessor on the training set and transform both train and test.

- Apply *SelectKBest* with *f_classif* to the preprocessed training data. Select the top 10 features. List the selected features and their F-scores.
- Apply *SelectFromModel* using a *GradientBoostingClassifier(n_estimators=100, max_depth=3, random_state=27)* on the preprocessed training data. List the selected features and their importance scores.
- Compute *permutation feature importance* (PFI) on the test set using the same GBC model. List the top 10 features by PFI.
- Compare the three feature sets: which features appear in both SelectKBest and SelectFromModel? Which appear in only one? How do the PFI rankings compare?
- Build three pipelines — (1) full features + GBC, (2) *SelectKBest(k=10)* + GBC, and (3) *SelectFromModel* + GBC — and compare their 5-fold cross-validated ROC AUC scores.

Analytical questions

1. How many features did each method select, and which features appear in both the filter and embedded sets?
2. Do the PFI rankings agree more with the filter selections or the embedded selections? Why?
3. Did feature selection improve, match, or hurt CV AUC compared to using all features? What does this tell you about the value of selection for this dataset?

4. Which features would you expect to matter most for churn based on domain knowledge (contract type, tenure, services), and do the methods confirm that expectation?
5. Which selection approach would you recommend for a production churn model on this dataset, and why?

Customer Churn – Case Study Answers

These answers assume you used the Telco Customer Churn dataset (7,043 rows, 21 columns), dropped *customerID*, converted *TotalCharges* to numeric, froze an 80/20 stratified split with **random_state = 27**, and used a *ColumnTransformer* preprocessing pipeline with *OneHotEncoder(drop='first')*. After encoding, the dataset has 30 features.

Q1. Feature counts and overlap

SelectKBest (f_classif, k=10) selected 10 features. The top five by F-score were: *tenure* (F = 786), *PaymentMethod_Electronic check* (F = 600), *InternetService_Fiber optic* (F = 594), *Contract_Two year* (F = 563), and *StreamingMovies_No internet service* (F = 305). The remaining five were other “No internet service” indicator variables and *InternetService_No*.

SelectFromModel (GBC) selected 7 features: *tenure* (importance 0.30), *InternetService_Fiber optic* (0.20), *PaymentMethod_Electronic check* (0.10), *Contract_Two year* (0.08), *TotalCharges* (0.07), *Contract_One year* (0.06), and *MonthlyCharges* (0.06).

The **overlap** is 4 features: *tenure*, *InternetService_Fiber optic*, *PaymentMethod_Electronic check*, and *Contract_Two year*. The filter method uniquely selected six “No internet service” indicators, while the embedded method uniquely selected *TotalCharges*, *MonthlyCharges*, and *Contract_One year*. This difference arises because f_classif measures individual feature–target association (the “no internet service” dummies are strongly associated with lower churn), while GBC importance reflects each feature’s contribution inside a multivariate model where correlated features share credit.

Q2. PFI agreement with filter vs embedded

PFI rankings agree more closely with the embedded selections. The top 5 PFI features were: *tenure* (PFI = 0.051), *MonthlyCharges* (0.022), *TotalCharges* (0.021), *Contract_Two year* (0.011), and *InternetService_Fiber optic* (0.006). These align with the 7 features selected by SelectFromModel. In contrast, the “No internet service” indicators that dominated the filter list appear low in PFI, because they are highly correlated with each other and with *InternetService_No* — shuffling one does not hurt the model much when the others remain intact. This is a practical illustration of why PFI can disagree with univariate F-scores in the presence of correlated features.

Q3. Did selection improve CV AUC

Feature selection **slightly hurt** CV AUC compared to using all features. The full-feature pipeline achieved **AUC = 0.8482 ± 0.0089**, SelectKBest(k=10) achieved *0.8360 ± 0.0076*, and SelectFromModel achieved *0.8373 ± 0.0094*. The differences are small but consistent across folds. This result is common when the feature count is already modest (30 features) and the model (GBC) handles irrelevant features well via its built-in feature importance weighting. Feature selection becomes more valuable when the feature-to-sample ratio is

high or when using models that are more sensitive to noise features (such as logistic regression or k-NN).

Q4. Domain expectations

Domain knowledge suggests that *tenure* (longer customers are less likely to churn), *contract type* (month-to-month contracts have higher churn), *internet service type* (fiber optic customers churn more, possibly due to pricing), and *payment method* (electronic check is associated with higher churn) should matter most. Both methods confirm these expectations: *tenure* is the top feature across all methods, and contract, internet, and payment features are consistently selected. The numeric charges features (*MonthlyCharges*, *TotalCharges*) are picked up by the embedded method and PFI but not by the filter method, which is a useful illustration of how univariate filters can miss features that are important in multivariate context.

Q5. Recommendation

For this dataset, the best recommendation is to **use all features without selection**, since the full-feature pipeline produced the highest CV AUC and the feature count (30) is small enough that selection offers no computational or interpretability benefit. If selection is required (for example, to reduce a deployed model's input requirements), SelectFromModel with GBC is the better choice because it selects a compact set of 7 features with only a modest AUC drop (0.8373 vs 0.8482) and aligns more closely with PFI and domain expectations than the filter method.

Case #2: Employee Attrition (Causal vs Predictive Paradigms)
This case uses the **Employee Attrition** dataset to practice the chapter's central theme: **choosing between causal and predictive feature selection**.

You will apply VIF-based removal with logistic regression (causal paradigm) and SelectFromModel with gradient boosting (predictive paradigm) to the same data, compare the resulting feature sets, and evaluate the tradeoff between interpretability and predictive power.

Dataset attribution: This dataset is widely distributed as an "Employee Attrition / HR Analytics" teaching dataset based on IBM HR sample data and is provided in this course as *Employee_Attrition.csv* (1,470 rows, 35 columns). See details on Kaggle.com The Employee Attrition dataset is available in the prior chapter if you need to reload it.

Prediction goal: Predict whether *Attrition* is *Yes* (employee leaves) or *No* (employee stays) using the remaining columns as predictors.

For reproducibility, use **random_state = 27** everywhere a random seed is accepted. Drop constant or identifier columns (*EmployeeNumber*, *EmployeeCount*, *Over18*, *StandardHours*) before analysis.

Tasks

- Load the dataset, drop *EmployeeNumber*, *EmployeeCount*, *Over18*, and *StandardHours*. Define *X* and *y* (where *y* = *Attrition*, mapped to 0/1). Identify numeric and categorical columns.
- Build a preprocessing pipeline using *ColumnTransformer*. Freeze an 80/20 stratified split with *random_state=27*. Fit and transform the training data.
- **Causal step 1 — VIF removal:** Compute VIF for all numeric features. Iteratively drop the feature with the highest VIF until all remaining features have VIF ≤ 5. Record which features were dropped.

- **Causal step 2 — Logistic regression:** Using *statsmodels*, fit a logistic regression on the VIF-cleaned numeric features. Report coefficients, standard errors, z-values, and p-values. Identify which features are statistically significant at $p < 0.05$.
- **Predictive approach:** Apply *SelectFromModel* with *GradientBoostingClassifier(n_estimators=100, max_depth=3, random_state=27)* on the full preprocessed training data (including categorical features). List all selected features.
- Compare the causal feature set (VIF-cleaned + statistically significant) with the predictive feature set (SelectFromModel). List the overlap, the features unique to each approach, and explain why the differences exist.
- Compute permutation feature importance on the test set and compare the PFI rankings with both feature sets.
- Compare pipeline performance using 5-fold cross-validated ROC AUC: (1) full features + GBC, (2) VIF-cleaned numeric + logistic regression, (3) SelectFromModel + GBC, and (4) RFECV-selected + GBC.

Analytical questions

1. Which feature(s) were dropped by VIF, and what collinearity does the removal reveal?
2. Which features were selected by the causal approach but not the predictive one, and vice versa? Why do the two paradigms disagree?
3. How do the pipeline CV AUC scores compare, and what does the causal model's lower AUC tell you about the tradeoff between interpretability and prediction?
4. If HR wants to understand *why* employees leave (to design interventions), which feature set and model is more appropriate?

5. If HR wants to *predict* who will leave next quarter (to flag for retention outreach), which feature set and model should they use?

Employee Attrition – Case Study Answers

These answers assume you used the Employee Attrition dataset (1,470 rows, 35 columns), dropped constant and identifier columns, froze an 80/20 stratified split with **random_state = 27**, and preprocessed with *StandardScaler* for numeric and *OneHotEncoder* for categorical features (44 features after encoding). The attrition rate is approximately 16.1% (237 Yes, 1,233 No).

Q1. VIF removal and collinearity

VIF analysis identified **JobLevel** (VIF = 10.99) as the feature exceeding the threshold of 5. After dropping *JobLevel*, all remaining VIF values fell below 5, with the next highest being *TotalWorkingYears* (VIF = 4.68) and *YearsAtCompany* (VIF = 4.56). *MonthlyIncome* dropped from VIF = 10.52 to VIF = 2.55 after removing *JobLevel*, confirming that these two features were the primary source of collinearity: *JobLevel* is essentially a proxy for *MonthlyIncome* (higher job levels correspond to higher pay). Removing one cleaned up the entire VIF structure.

Q2. Causal vs predictive feature sets

The *causal approach* (VIF-cleaned + $p < 0.05$) selected 13 features: *Age*, *DistanceFromHome*, *EnvironmentSatisfaction*, *JobInvolvement*, *JobSatisfaction*, *MonthlyIncome*, *NumCompaniesWorked*, *StockOptionLevel*,

TotalWorkingYears, *YearsAtCompany*, *YearsInCurrentRole*, *YearsSinceLastPromotion*, and *YearsWithCurrManager*.

The *predictive approach* (SelectFromModel with GBC) selected 16 features, including both numeric and one-hot encoded categorical features: *MonthlyIncome* (importance 0.117), *OverTime_Yes* (0.113), *Age* (0.091), *YearsAtCompany* (0.050), *DistanceFromHome* (0.048), *HourlyRate* (0.043), *StockOptionLevel* (0.043), *JobSatisfaction* (0.041), *EnvironmentSatisfaction* (0.036), *DailyRate* (0.036), *MonthlyRate* (0.035), *JobRole_Sales Executive* (0.033), *TotalWorkingYears* (0.029), *NumCompaniesWorked* (0.028), *JobLevel* (0.026), and *YearsWithCurrManager* (0.024).

The **overlap** is 10 features: *Age*, *DistanceFromHome*, *EnvironmentSatisfaction*, *JobSatisfaction*, *MonthlyIncome*, *NumCompaniesWorked*, *StockOptionLevel*, *TotalWorkingYears*, *YearsAtCompany*, and *YearsWithCurrManager*. **Causal only**: *JobInvolvement*, *YearsInCurrentRole*, *YearsSinceLastPromotion* (these have small but statistically significant coefficients). **Predictive only**: *DailyRate*, *HourlyRate*, *JobLevel*, *JobRole_Sales Executive*, *MonthlyRate*, *OverTime_Yes*. The disagreements make sense: the causal model dropped *JobLevel* for collinearity and cannot access categorical features like *OverTime*, while the predictive model included *OverTime_Yes* as its second most important feature because it is strongly predictive regardless of collinearity.

Q3. Pipeline CV AUC comparison

Full features + GBC achieved **AUC = 0.8040 ± 0.0316**. VIF-cleaned numeric + logistic regression achieved *0.7558 ± 0.0470*. SelectFromModel + GBC achieved *0.7939 ± 0.0413*. RFECV-selected + GBC achieved *0.8121 ± 0.0330*.

The causal model's lower AUC illustrates the fundamental tradeoff: VIF removal sacrificed predictive power to gain interpretability, and restricting to numeric features excluded *OverTime*, which was the single most important predictive feature (PFI = 0.057). The RFECV pipeline slightly outperformed the full pipeline, suggesting that removing a few uninformative features improved generalization on this small dataset.

Q4. Understanding why employees leave

If the goal is to understand *why* employees leave — to design interventions such as adjusting compensation, improving satisfaction, or targeting retention programs — the **causal approach** (VIF-cleaned logistic regression) is more appropriate. Its coefficients can be interpreted as: "holding all else equal, a one-unit increase in this feature is associated with this change in the log-odds of attrition." For example, the negative coefficient on *JobSatisfaction* (coeff = −0.376, $p < 0.001$) tells HR that higher job satisfaction is associated with lower attrition, which is actionable. The predictive model's importance scores tell you *which* features matter but not *how* or *why*.

Q5. Predicting who will leave

If the goal is to *predict* who will leave next quarter, the **RFECV-selected + GBC pipeline** is the best choice (AUC = 0.8121). It includes both numeric and categorical features (notably *OverTime_Yes* and *MaritalStatus_Single*), uses a nonlinear model that captures interactions, and achieved the highest cross-validated ranking performance. The full-feature pipeline is a close second (AUC = 0.8040). The causal logistic regression, while interpretable, is not the right tool here because its lower AUC means it will miss more at-risk employees when used for flagging and prioritization.

Case #3: Support Ticket Priority (Pipeline Integration and Joint Tuning)
This case uses a **support ticket priority** dataset to practice **feature selection inside pipelines** and **joint hyperparameter tuning** in a multiclass setting. Your goal is to compare filter methods, evaluate embedded selection, and use *GridSearchCV* to simultaneously tune the number of selected features and the model's hyperparameters.

Dataset attribution: The dataset file for this case is *Support_tickets.csv* (50,000 rows, 34 columns). See details on Kaggle.com The Support Ticket Priority dataset is available in the prior chapter if you need to reload it.

Prediction goal: Predict *priority* (*Low*, *Medium*, *High*). Treat the classes as nominal (not ordered) and use standard multiclass classification metrics.

Recommended feature set: Use a mix of numeric and categorical predictors. Drop *ticket_id*, *company_id*, and all columns whose names end with *_cat* (duplicate numeric encodings of categorical features). Keep human-readable categorical columns and numeric operational columns.

For reproducibility, use **random_state = 27** everywhere a random seed is accepted.

Tasks

- Load the dataset, drop *ticket_id*, *company_id*, and all *_cat* columns. Define *X* and *y* (where *y = priority*). Report the class distribution.
- Build a preprocessing pipeline using *ColumnTransformer*. Freeze an 80/20 stratified split with *random_state=27*. Fit and transform the training data.
- Apply *SelectKBest* with *f_classif* (top 15) and *mutual_info_classif* (top 15) on the preprocessed training data. List the top 15 for each and

compare the overlap.

- Apply *SelectFromModel* with *GradientBoostingClassifier(n_estimators=100, max_depth=3, random_state=27)*. List the selected features and their importance scores.
- Compute permutation feature importance on the test set using accuracy as the scorer. List the top 15 features by PFI.
- Compare three pipelines — (1) full features + GBC, (2) *SelectKBest(k=15)* + GBC, and (3) *SelectFromModel* + GBC — using 5-fold cross-validated accuracy and log loss.
- Use *GridSearchCV* to jointly tune *SelectKBest k* (5, 10, 15, 20) and GBC *max_depth* (3, 5) inside a pipeline. Report the best parameter combination and its CV accuracy.

Analytical questions

1. Do *f_classif* and *mutual_info_classif* agree on the top features? Where do they disagree and why might that happen?
2. Did feature selection improve or hurt CV accuracy and log loss compared to using all features?
3. What is the best combination of *k* and *max_depth* from joint tuning, and how does it compare to the full-feature pipeline?
4. Which features dominate PFI, and are they the same features that dominate GBC importance?
5. Given the multiclass nature of this problem, which selection method would you recommend for a production support-ticket system?

Support Ticket Priority – Case Study Answers

These answers assume you used the Support Ticket Priority dataset (50,000 rows), dropped *ticket_id*, *company_id*, and all *_cat* columns, froze an 80/20 stratified split with **random_state = 27**, and preprocessed with *StandardScaler* for numeric and *OneHotEncoder(handle_unknown='ignore')* for categorical features (53 features after encoding). The class distribution is: *Low* 25,000 (50.0%), *Medium* 17,500 (35.0%), *High* 7,500 (15.0%).

Q1. Agreement between f_classif and mutual_info_classif

The two scoring functions show **strong agreement**: 14 of the top 15 features are shared. Both methods rank *org_users*, *customers_affected*, *company_size_Large*, *description_length*, *downtime_min*, and *customer_tier_Basic* among the top features. The one disagreement is that f_classif includes *product_area_auth* while mutual_info_classif includes *industry_gaming* in the 15th slot. This high overlap is expected because the dataset has strong, roughly linear feature–target relationships; mutual information captures nonlinear patterns as well, but when the signal is mostly linear, the two methods converge. The slight difference may reflect a weak nonlinear association between *industry_gaming* and priority that f_classif's linear test does not detect.

Q2. Did selection improve or hurt performance

Feature selection **hurt** both accuracy and log loss compared to using all features. The full-feature pipeline achieved **accuracy = 0.9384 ± 0.0019** and **log loss = 0.2044 ± 0.0029**. SelectKBest(k=15) dropped to *0.9160* ± *0.0030* accuracy and *0.2443* ± *0.0060* log loss. SelectFromModel (which selected only 4 features) dropped further to *0.9117* ± *0.0044* accuracy and *0.2471* ± *0.0058* log loss. With 50,000 rows and 53 features, the dataset has a very favorable sample-to-feature ratio, and the GBC model handles irrelevant

features well. Aggressive selection removes features that contribute small but meaningful signal (such as *has_runbook*, *payment_impact_flag*, and *reported_by_role* categories), resulting in worse performance.

Q3. Best joint tuning combination

The best combination from GridSearchCV was **k = 20** and **max_depth = 5**, achieving CV accuracy of **0.9356**. This is close to but still below the full-feature pipeline (0.9384). Increasing *k* consistently improved accuracy (k=5 scored only 0.687, k=10 scored 0.914, k=15 scored 0.916–0.918, k=20 scored 0.928–0.936), confirming that this dataset benefits from retaining more features. Deeper trees (max_depth=5) consistently outperformed shallower trees (max_depth=3), reflecting meaningful interaction effects in the data. The joint tuning result suggests that if selection is required, use at least k=20 features.

Q4. PFI vs GBC importance rankings

PFI and GBC importance **agree closely** on the top 4 features: *customers_affected* (PFI = 0.425, GBC importance = 0.569), *downtime_min* (PFI = 0.274, importance = 0.290), *error_rate_pct* (PFI = 0.106, importance = 0.077), and *customer_tier_Enterprise* (PFI = 0.051, importance = 0.025). These four features dominate both rankings by a large margin. The agreement is strong because these features have clear, direct relationships with ticket priority, and there is limited collinearity among them. Below the top 4, PFI reveals that *reported_by_role_c_level* (PFI = 0.018) and *has_runbook* (PFI = 0.011) have meaningful impact despite low GBC importance scores, illustrating that GBC importance can underweight categorical features with many levels.

Q5. Recommendation for production

For a production support-ticket system, the recommendation is to **use all features without selection**, since the full-feature pipeline achieved the highest accuracy (0.9384) and lowest log loss (0.2044) and the feature count (53) is manageable. If feature reduction is required for operational reasons (reducing the number of inputs that must be collected at ticket creation time), use *SelectKBest with k=20* and *max_depth=5*, which preserves 93.6% accuracy. Avoid aggressive selection ($k \leq 10$) because ticket priority depends on an interaction of operational severity features (*customers_affected*, *downtime_min*, *error_rate_pct*) and contextual features (*customer_tier*, *reported_by_role*, *has_runbook*) that individually have modest importance but collectively contribute meaningful signal.

16.11Assignment

Complete the assignment below.

This assessment can be taken online.